Gutar Gen 6: matematyka wielogłosowej transkrypcji gitary z dźwięku do nut i tabulatury

Wyjaśnienie wszystkich wzorów od zera, a potem pełny opis modelu w formie pracy naukowej

Projekt Gutar (autor do uzupełnienia)

5 listopada 2026

Spis treści

Streszczenie **2**

Jak czytać ten dokument **3**

Plan nauki (jak się uczyć z tego dokumentu) **3**

CZĘŚĆ I. MATEMATYKA KROK PO KROKU **4**
M0. Zapis matematyczny: słownik symboli i funkcji . . . 4
M1. Dźwięk jako ciąg liczb, próbkowanie . . . 10
M2. Wysokość dźwięku: Hz, półtony, centy, MIDI . . . 11
M3. Alikwoty (harmoniczne) . . . 13
M4. Transformata Fouriera: z drgań do widma . . . 14
M5. HCQT: widmo „ułożone” harmonicznie . . . 17
M6. Konwolucja i jej odmiany . . . 19
M7. Funkcje nieliniowe: sigmoid, softmax, GELU . . . 21
M8. Normalizacja warstw, połączenia resztkowe, dropout . . . 23
M9. Uwaga (*attention*) . . . 24
M10. Funkcje straty: jak model „wie”, że się myli . . . 26
M11. Optymalizacja: jak sieć się uczy . . . 27
M12. Dekodowanie: od map prawdopodobieństw do listy nut . . . 29
M13. Miary jakości . . . 30
M14. Jak to wszystko się łączy (mapa myślowa) . . . 32

CZĘŚĆ II. PRACA NAUKOWA **33**

1. Wstęp **33**

2. Prace pokrewne **33**

3. Dane i etykiety **34**
3.1. Zbiory . . . 34
3.2. Mapy celów . . . 34
3.3. Dostępność etykiet . . . 35

4. Cechy wejściowe **35**

5. Architektura sieci **36**
5.1. Przepływ tensorów . . . 36

5.2. Blok harmoniczny . . . 36
5.3. Zejście do półtonów i osadzenie wysokości . . . 37
5.4. Uwaga w czasie . . . 38
5.5. Uwaga między wysokościami (nowość) . . . 38
5.6. Warstwa kontekstu i pole widzenia . . . 38
5.7. Głowice . . . 39

6. Funkcja straty **39**

7. Uczenie **40**

8. Dekoder **40**

9. Protokół ewaluacji **42**

10. Wyniki **43**
10.1. Przebieg uczenia . . . 43
10.2. Test: model końcowy (epoka 81), 700 nagrań, makro . . . 43
10.3. GuitarSet (prawdziwa gitara, gracze 03–05) . . . 43
10.4. Zróżnicowanie instrumentów (sesja 1, makro F1 nut) . . . 44
10.5. Szybkość . . . 44

11. Dyskusja i ograniczenia **44**

12. Podsumowanie **46**

Najważniejsze myśli (podsumowanie dla uczącego się) **46**

Literatura **46**

Dodatek A. Skąd się bierze 249 219 parametrów **47**

Dodatek B. Ściąga wzorów (jedną linijką) **47**

Streszczenie

Opisujemy model **Gen 6** projektu Gutar: sieć neuronową, która z nagrania gitary (także z innych instrumentów) odtwarza **listę nut** (wysokość, początek, koniec) oraz **tabulaturę** (na której strunie i którym progiem zagrano daną nutę). Wejściem jest harmonicznie ułożone widmo (HCQT, 9 kanałów × 159 binów co 1/3 półtonu, krok 23 ms). Sieć ma 249 219 parametrów: trzon konwolucyjny z trzema *blokami harmonicznymi*, które patrzą jednocześnie na całą serię alikwotów kandydata na wysokość, oraz dwie warstwy kontekstu z *uwagą w czasie* (w obie strony, okno ±1 s) i *uwagą między wysokościami* (oktawy, kwinty, akord jako całość). Pięć głowic przewiduje: początek, trwanie, koniec, głośność i tabulaturę. Uczono ją łącznie 97 epok na ok. 56 godzinach nagrań (głównie syntetycznych, z domieszką prawdziwej gitary z GuitarSet), a nuty wydobywa deterministyczny dekoder z dostrojonymi progami.

Na 700 nagraniach testowych (180 GuitarSet + 520 syntetycznych utworów, nieużytych w uczeniu) model osiąga makro-F1 nut **86,6 %** i F1 nut z poprawnym końcem **70,0 %**. Na GuitarSet (gracze 03–05) F1 nut wynosi 87,2 % (domyślny dekoder) wobec 79,5 % dla poprzedniej generacji.

Główną część dokumentu stanowi jednak wyjaśnienie **całej matematyki** użytej w modelu, tak aby każdy wzór dało się zrozumieć bez wcześniejszej wiedzy: od częstotliwości i decybeli, przez

transformatę Fouriera, konwolucję i uwagę (*attention*), po funkcje strat, optymalizator AdamW i miary jakości.

Jak czytać ten dokument

- **Część I** (rozdziały M0–M14) uczy matematyki **po kolei**; M0 to słownik wszystkich symboli i funkcji, do którego warto wracać, gdy napotkasz nieznany znak, każdy wzór ma trzy elementy: *intuicję* (po ludzku), *wzór* i *przykład liczbowy*. Wszystkie liczby w przykładach policzono naprawdę.
- **Część II** (rozdziały 1–12) to właściwa praca naukowa. Na każdy wzór wskazuje, w którym rozdziale Części I jest wyjaśniony (np. „zob. M9”).
- **Rysunki** (25 szt.) są generowane skryptem `tools/make_figures_paper.py`: rysunek 4 to prawdziwe wyjście kodu cech, rysunki z wynikami modelu pochodzą z plików w `docs/wyniki/`, a rysunki oznaczone w podpisie jako poglądowe (m.in. dekoder, premia uwagi, optymalizacja, kalibracja, bootstrap) służą tylko do zrozumienia metody i nie zawierają danych z modelu.
- **Przykłady liczbowe** są oznaczone słowem „Przykład” i można je sprawdzić ręcznie.
- **Dodatki**: tabela wszystkich 249 219 parametrów (A), ściąga symboli (B).
- Odwołania do kodu mają postać `plik:funkcja` i dotyczą katalogu `tools/gen6/`.

Oznaczenia, które wracają w całym tekście:

symbol	znaczenie	wartość w modelu
f_s	częstotliwość próbkowania	11 025 Hz
T	liczba klatek czasowych nagrania	zależy od długości
S	liczba półtonów (wysokości)	53 (MIDI 36–88)
P	liczba binów częstotliwości	159 ($= 3S$)
C	liczba kanałów wejścia (HCQT)	9
H, D	liczba kanałów w trzonie / w warstwach kontekstu	48 / 64
t, s, p	indeks klatki, półtonu, binu	

Plan nauki (jak się uczyć z tego dokumentu)

Idź po kolei, a przy każdym rozdziale Części I rób trzy rzeczy: (1) przeczytaj ramkę „Po co nam to?”, (2) przeczytaj wyjaśnienie i **sam przelicz przykład na kartce**, (3) rozwiąż „Sprawdź siebie”. Gdy odpowiedzi się zgadzają, idziesz dalej.

krok	rozdział	co umiesz po nim	czas (orientacyjnie)
0	M0	czytać wszystkie znaki	20 min
1	M1–M3	dźwięk jako liczby, wysokość i alikwoty	40 min
2	M4–M5	skąd bierze się wejście sieci (HCQT)	60 min

krok	rozdział	co umiesz po nim	czas (orientacyjnie)
3	M6–M9	konwolucja, funkcje, normalizacja, uwaga: „wnętrze” sieci	90 min
4	M10–M11	jak sieć się uczy	45 min
5	M12–M14	dekoder, miary jakości i całość	45 min
6	Część II	czytasz właściwą pracę i rozumiesz każdy wzór (przy każdym rozdziale jest „Po ludzku” i „Powtórka”)	90 min

Jedno zdanie o całym modelu: *dźwięk zamieniamy na obraz „kto gra w jakiej wysokości i kiedy” (HCQT), sieć uczy się czytać ten obraz (blok harmoniczny + uwaga), a prosty dekoder zamienia jej odpowiedzi na listę nut i tabulaturę.*

CZĘŚĆ I. MATEMATYKA KROK PO KROKU

M0. Zapis matematyczny: słownik symboli i funkcji

> **Po co nam to?** Wzory są zapisane „alfabetem” matematyki. Dopóki nie znasz liter, nie przeczytasz zdania. Ten rozdział to alfabet: nie musisz go uczyć się na pamięć, wystarczy wracać do niego, gdy trafisz na nieznany znak.

Po co ten rozdział. Zanim wejdziemy w dźwięk, wyjaśniamy **każdy znak**, który potem się pojawia. Jeśli w dalszej części trafisz na symbol, którego nie znasz, wróć tutaj (albo do Dodatku B). Każdy symbol ma: zapis, odczyt po polsku, co robi i mały przykład.

Działania i funkcje podstawowe

zapis	jak czytać	co znaczy	przykład
$a \cdot b$, $a \times b$	a razy b	mnożenie (kropka w tekście, $\times$ także w *wymiarach*: $T \times 9$)	$3 \cdot 4 = 12$
$\frac{a}{b}$, a/b	a przez b	dzielenie	$\frac{1}{4} = 0{,}25$
a^n	a do potęgi n	a pomnożone n razy; $a^{-n} = 1/a^n$; $a^{1/2} = \sqrt{a}$	$2^3 = 8,\ 2^{-1} = 0{,}5$
$\sqrt{a}$	pierwiastek z a	liczba, której kwadrat to a	$\sqrt{16} = 4$
$e,\ e^x,\ \exp(x)$	„e do x”	$e \approx 2{,}718$ (liczba Eulera); $\exp(x)$ to inny zapis e^x; rośnie bardzo szybko i jest zawsze dodatnia	$e^0 = 1,\ e^1 = 2{,}72$

zapis	jak czytać	co znaczy	przykład
$\ln x$	logarytm naturalny	odwrotność e^x: $\ln(e^x) = x$. Mała liczba (np. 0,1) daje dużą ujemną wartość	$\ln 1 = 0,\ \ln 0{,}1 = -2{,}30$
$\log_b x$	logarytm przy podstawie b	„do jakiej potęgi podnieść b, by dostać x”; $\log_2$ — podstawa 2 (oktawy!), $\log_{10}$ — podstawa 10 (decybele)	$\log_2 8 = 3,\ \log_{10} 1000 = 3$
$\sin,\ \cos,\ \pi$	sinus, cosinus, pi	funkcje fal (sin i cos przebiegają od -1 do 1 i powtarzają się co 2π); $\pi \approx 3{,}14159$	$\cos 0 = 1,\ \sin(\pi/2) = 1$
i (w $e^{-2\pi i\, kn/N}$)	jednostka urojona	$i^2 = -1$; liczby zespolone zapisują „wielkość i fazę” fali	$e^{i\phi} = \cos\phi + i\sin\phi$
$\lvert x\rvert$	wartość bezwzględna (moduł)	odległość od zera, bez znaku; dla liczby zespolonej — jej „wielkość”	$\lvert -3\rvert = 3$
$\lVert g\rVert_2$	norma euklidesowa	„długość wektora”: $\sqrt{g_1^2 + g_2^2 + \cdots}$	$\lVert(3,4)\rVert_2 = 5$
$\lfloor x\rfloor,\ \lceil x\rceil$	podłoga, sufit	zaokrąglenie w dół / w górę do całkowitej	$\lfloor 121{,}3\rfloor = 121,\ \lceil 123{,}6\rceil = 124$
$\mathrm{round}(x)$	zaokrąglenie	do najbliższej całkowitej	$\mathrm{round}(35{,}07) = 35$
$\min(a,b),\ \max(a,b)$	mniejsza, większa z dwóch	$\max_{k\in A} x_k$ — największa wartość x_k po wszystkich k ze zbioru A	$\max(3,7) = 7$
$\arg\max_i x_i$	argument maksimum	**numer** i, dla którego x_i jest największe (nie sama wartość!)	dla $x = (0{,}1; 0{,}7; 0{,}2)$: $\arg\max = 2$
$\mathrm{clip}(x,a,b)$	przycięcie	$\min(\max(x,a),b)$: wartości poniżej a zamienia na a, powyżej b na b	$\mathrm{clip}(1{,}7;\ 0;\ 1{,}3) = 1{,}3$
$\mathrm{erf}(x)$	funkcja błędu	$\frac{2}{\sqrt{\pi}}\int_0^x e^{-u^2}du$; „S-kształtna” od -1 do 1; potrzebna tylko do definicji GELU	$\mathrm{erf}(0) = 0$
$\Pr(A)$	prawdopodobieństwo A	liczba z $[0,1]$: jak często zdarza się A	$\Pr(\text{orzeł}) = 0{,}5$

Sumy i indeksy

zapis	znaczenie	przykład
$\sum_{n=1}^{N} a_n$	**suma**: $a_1 + a_2 + \cdots + a_N$. Litera pod $\sum$ (tu n) to licznik, który „przebiega” podane wartości	$\sum_{n=1}^{3} n^2 = 1 + 4 + 9 = 14$
$\sum_{j} a_{ij}$	suma po wszystkich j (zakres jest oczywisty z kontekstu); i zostaje stałe	$\sum_j a_{ij} = 1$ dla wag uwagi
$x_i,\ x[n],\ x_{t,s}$	**element** ciągu / tablicy o numerze (indeksie) i, n, albo parze (t, s)	$x = (5, 8, 2) \Rightarrow x_2 = 8$
$x[0..N-1]$, ..., $\cdots$	zakres „od 0 do $N-1$”; wielokropek = „i tak dalej”	$1, 2, \ldots, 5$
$h^{(0)}$, $x^{(c)}$	górny indeks w nawiasie to **etykieta** (numer kanału, warstwy), nie potęga	$h^{(2)}$ — wyjście 2. warstwy

Zbiory, wymiary, tensory

zapis	znaczenie	przykład
$x \in A$	x **należy do** zbioru A	$3 \in \{1, 2, 3\}$
$\{1, \ldots, 6\}$, $[0, 1]$, $(0, 1)$	zbiór liczb; $[a, b]$ — przedział z końcami, (a, b) — bez końców	$\sigma(z) \in (0, 1)$
$\mathbb{R}$	wszystkie liczby rzeczywiste	
$\mathbb{R}^d$	**wektor** d liczb rzeczywistych	$z \in \mathbb{R}^{64}$: 64 liczby
$\mathbb{R}^{T \times 9 \times 159}$	**tensor** (tablica wielowymiarowa) o podanych rozmiarach wzdłuż kolejnych osi: T klatek, 9 kanałów, 159 binów	to nasze wejście X
$[B, C, T, P]$	zapis **kształtu** tensora w nawiasach kwadratowych (to samo co $\mathbb{R}^{B \times C \times T \times P}$)	$[B, 48, T, 159]$
$\{0, 1\}^5$	pięcioelementowy ciąg, każdy element to 0 albo 1	wektor dostępności etykiet
$\mathcal{B}_b$, $\mathcal{V}$, $\mathcal{N}$	**zbiory** (kaligrafia $\mathcal{A}$ oznacza zbiór lub „wielką” wielkość, np. stratę $\mathcal{L}$)	$\mathcal{V}$ — zbiór par z etykietą
$\mathbf{p}$	pogrubiona litera = wektor	rozkład softmax $\mathbf{p}$
$q^\top k,\ W^\top$	$\top$ (**transpozycja**): zamiana wiersza na kolumnę. $q^\top k = \sum_l q_l k_l$ to iloczyn skalarny	$(1, 2)^\top (3, 4) = 3 + 8 = 11$
$[\,u\,;v\,]$, $u \,\|\, v$	**sklejenie** (konkatenacja) wektorów lub tensorów: ustawienie jednego za drugim	$(1, 2) \| (3) = (1, 2, 3)$

Porównania i strzałki logiczne

zapis	czytaj	zapis	czytaj
$=$	równa się	$\approx$	w przybliżeniu
$\leq,\ \geq$	mniejsze lub równe, większe lub równe	$<,\ >$	mniejsze, większe
$\pm$	plus minus	∞	nieskończoność ($-\infty$ — „bardzo bardzo ujemna”)
$\Rightarrow$	„zatem” (wynika)	$\Leftrightarrow$	„wtedy i tylko wtedy”
$\rightarrow$	„przechodzi w”, „daje”; też $x \rightarrow 0$ — „x zbliża się do 0”	$\leftarrow$	„podstaw”: $\theta \leftarrow \theta - \eta g$ znaczy „nową wartość θ ustaw na $\theta - \eta g$”
$*$ (w i^*)	gwiazdka = „wybrany/optymalny” (nie mnożenie)	$\overbrace{\cdots}$	klamra z podpisem: „ta część wyrażenia to …”

Dekoracje liter (kreska, daszek, falka)

zapis	znaczenie
$\hat{m},\ \hat{v},\ \hat{v}_t$	daszek: **wartość po korekcie** lub **przewidziana** przez model (np. $\hat{v}$ — przewidziana głośność; $\hat{m}_t$ — pęd po korekcie obciążenia)
$\bar{y},\ \bar{p}$	kreska: **średnia**
$\tilde{h}$	falka: **wersja znormalizowana/przekształcona** zmiennej h
$x_t,\ x_{t-1}$	t — numer **kroku/klatki** (z kontekstu wiadomo, czy chodzi o krok uczenia, czy klatkę audio)

Pochodne i gradient (pełne wyjaśnienie w M11)

zapis	znaczenie
$\frac{df}{dx}$	**pochodna**: „jak szybko f rośnie, gdy zwiększamy x” (nachylenie wykresu). Dla $f = x^2$: $\frac{df}{dx} = 2x$
$\frac{\partial \mathcal{L}}{\partial \theta_i}$	**pochodna cząstkowa**: jak zmienia się $\mathcal{L}$, gdy ruszamy *tylko* θ_i, a resztę zostawiamy. ∂ („d okrągłe”) zamiast d znaczy „zmiennych jest więcej niż jedna”
$\nabla_\theta \mathcal{L}$	**gradient** (nabla): wektor wszystkich $\frac{\partial \mathcal{L}}{\partial \theta_i}$; wskazuje, w którą stronę $\mathcal{L}$ rośnie najszybciej
Δ	**różnica / odstęp**: np. $\Delta f = f_s/N$ (odstęp binów), $\Delta = 256/f_s$ (czas jednej klatki)

Litery greckie i ich role (uwaga na powtarzające się litery!)

Greckich liter jest mało, więc w różnych miejscach oznaczają różne rzeczy. Za każdym razem, gdy wprowadzamy literę, mówimy, co znaczy. Ściąga:

litera	znaczenia w tym dokumencie
σ	**(a) sigmoid** $\sigma(z) = 1/(1+e^{-z})$, gdy ma nawias z argumentem; **(b) odchylenie standardowe** σ i wariancja σ^2 w LayerNorm
θ	**(a) wszystkie parametry (wagi) sieci**, w optymalizacji (M11); **(b) próg** dekodera: $\theta_{\text{on}}, \theta_{\text{fr}}, \theta_{\text{off}}, \theta_{\text{start}}, \theta_{\text{merge}}$, w rozdz. 8 (nigdy w tym samym zdaniu)
η	tempo uczenia (*learning rate*), η_0 — wartość początkowa
λ	współczynnik rozpadu wag (*weight decay*) w AdamW
β	**(a)** β_1, β_2 — stałe wygładzania w Adamie (0,9 i 0,999); **(b)** β_i — wyuczone przesunięcie w LayerNorm; **(c)** $\beta_{i,f}$ — wyuczona premia za próg w głowicy tabulatury
γ	wyuczona skala w LayerNorm (w stracie ogniskowej γ byłoby wykładnikiem, ale w modelu $= 0$, więc nieużywane)
α	waga klasy pozytywnej w BCE ($\alpha = 4$ dla początków)
ε	**bardzo mała liczba** (np. 10^{-8}) dodawana, żeby nie dzielić przez zero
μ	**średnia** wektora (LayerNorm); w stracie głośności μ to maska
τ	czas w **sekundach** (np. τ_t — środek klatki t); odróżnia się od t (numer klatki)
$\phi,\ \Phi$	ϕ — kąt/faza w $e^{-i\phi}$; $\Phi(z)$ — **dystrybuanta rozkładu normalnego** (pole pod „dzwonem” do z; $\Phi(0) = 0{,}5$)
$\ell,\ \mathcal{L}$	ℓ — **strata jednego elementu**; $\mathcal{L}$ — strata **łączna** (średnia po wielu)
$\mathcal{U}(a, b)$	**rozkład jednostajny**: losowa liczba z $[a, b]$, każda równie prawdopodobna
$\emptyset$	„nic” (puste); w tabulaturze oznacza klasę „struna milczy”

Powtarzające się litery łacińskie (wymiary i oznaczenia)

litera	znaczenie
f_s, N, M	częstotliwość próbkowania; długość ramki FFT w próbkach (N) i po dopełnieniu zerami ($M = 4N$)
$T,\ S,\ P,\ C,\ B$	liczba: klatek czasu, półtonów (53), binów (159), kanałów wejścia (9), nagrań w paczce (*batch*)
$H,\ D$	liczba kanałów w trzonie (48) i w warstwach kontekstu (64)
$n_h,\ d_h$	liczba głów uwagi (4) i wymiar jednej głowy ($d_h = D/n_h = 16$)

litera	znaczenie
$t,\ s,\ p$	numer klatki, półtonu, binu; m — numer nuty MIDI; f — częstotliwość (Hz) albo, w tabulaturze, numer progu
h	**uwaga:** h to (a) numer alikwotu $h = 1, 2, 3 \ldots$ w rozdz. M3–M5, (b) wektor cech sieci $h_{t,s}$ od rozdz. M8 — z kontekstu wiadomo który
W	**bez indeksu: pół-okno uwagi w czasie (**$W = 43$ **klatek)**; z indeksem ($W_h, W_Q, \ldots$): macierz wag
$k,\ K$	k: numer binu FFT, albo przesunięcie k_h w binach, albo klucz k_j w uwadze; K: zbiór przesunięć $\{-36, 0, \ldots, 93\}$ albo rozmiar filtra $K_t \times K_p$
$q,\ k,\ v$	zapytanie, klucz, wartość w uwadze (M9)
$x,\ y,\ z$	ogólne: wejście, wyjście, wektor roboczy (z także jako logit)
$X,\ A$	X — tensor wejściowy HCQT; A — amplituda
$L,\ L_{\text{ref}}$	poziom w decybelach i poziom odniesienia (M5); δ — długość nuty (inna rzecz!)
a_{ij}	waga uwagi z elementu i do j; a_{head} — dostępność etykiety danej głowicy
$b,\ c$	obciążenia (stałe dodawane po mnożeniu przez wagi); b_{j-i} — premia zależna od odległości
o_i	wysokość MIDI pustej struny i (40, 45, 50, 55, 59, 64)
$p_{\text{on}}, p_{\text{fr}}, p_{\text{off}}$	przewidziane prawdopodobieństwa początku, trwania („brzmi”), końca; y_{on} itd. — etykiety prawdy

Nazwy funkcji i operacji, które pojawiają się w opisie sieci

Pisane prostą czcionką, bo to nazwy, nie zmienne. Każda ma dalej pełne wyjaśnienie w wskazanym rozdziale.

nazwa	co robi (jednym zdaniem)	gdzie
Conv, DW	konwolucja; DW = konwolucja głębokościowa (każdy kanał osobno)	M6
$\text{shift}_k(h)$	**przesunięcie** osi binów o k pozycji: $h[p + k]$, a poza zakresem zera	M3, 5.2
σ, softmax, GELU	funkcje nieliniowe	M7
LN (LayerNorm), LN_{kan}	normalizacja po kanałach	M8
Drop	dropout: losowe zerowanie elementów	M8
FF, MLP	sieć z dwóch warstw liniowych i GELU w środku: $W_2\,\text{GELU}(W_1 z)$	5.6, 5.7
TA, PA	uwaga w czasie (*time attention*), uwaga między wysokościami (*pitch attention*)	M9, 5.4–5.5

nazwa	co robi (jednym zdaniem)	gdzie
$\mathrm{sg}(\cdot)$	*stop-gradient*: wartość przepuszczana do przodu, ale „niewidoczna” dla gradientu	5.7
clip	przycięcie	M5

Ważne o macierzach W. W (z indeksami: $W_h, W_Q, W_K, W_V, W_O, W_1, W_2$) zawsze oznacza **wyuczoną macierz wag**: tabelę liczb, przez którą mnożymy wektor, np. $y = Wx$ znaczy $y_i = \sum_j W_{ij} x_j$. To właśnie te liczby (razem z obciążeniami b, c) są uczone: razem 249 219 sztuk.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Ile wynosi $\arg\max$ dla $x = (2, 9, 4)$? **Odp.:** 2, bo największa wartość (9) stoi na drugim miejscu. max dałby 9.
2. Co znaczy $\theta \leftarrow \theta - \eta g$? **Odp.:** nową wartość θ ustaw na starą pomniejszoną o η razy g.
3. Co mówi kształt $[B, T, 53, 64]$? **Odp.:** B nagrań, każde ma T klatek, w każdej 53 półtony, a każdy półton opisuje 64 liczby.

Najczęstszy błąd: mylenie wartości z jej numerem (max kontra $\arg\max$) oraz czytanie $h^{(2)}$ jako „h do kwadratu” (nawias w indeksie górnym to etykieta).

Co zapamiętać: Znaki są tylko skrótami. Każdy z nich da się wypowiedzieć zwykłym zdaniem.

M1. Dźwięk jako ciąg liczb, próbkowanie

Po co nam to? Komputer nie „słyszy”: widzi ciąg liczb. Zanim cokolwiek policzymy, musimy wiedzieć, co te liczby znaczą i ile ich jest na sekundę.

Intuicja. Mikrofon mierzy ciśnienie powietrza. Komputer nie zapisuje ciągłej krzywej, tylko **próbki**: ciśnienie mierzone co stały ułamek sekundy.

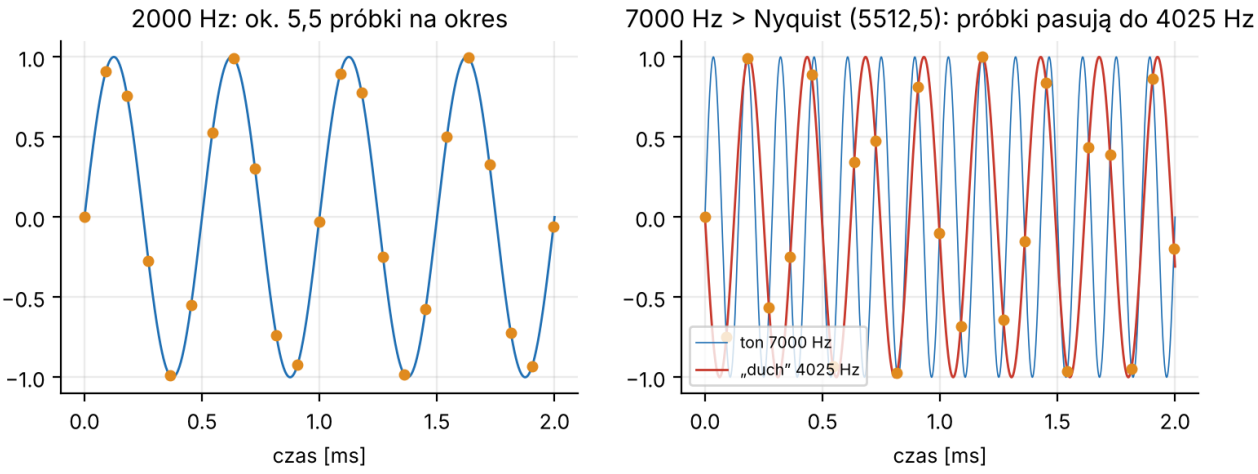
Wzór. Sygnał ciągły $x(\tau)$ (czas τ w sekundach) zamieniamy na ciąg

$$x[n] = x\left(\frac{n}{f_s}\right), \qquad n = 0, 1, 2, \ldots$$

gdzie f_s to liczba próbek na sekundę. Najwyższa częstotliwość, jaką da się jeszcze poprawnie zapisać, to **częstotliwość Nyquista** $f_s/2$ (twierdzenie o próbkowaniu): szybsze drgania „zawijają się” i udają wolniejsze.

Przykład. Gen 6 zmienia próbkowanie każdego nagrania na $f_s = 11\,025$ Hz. Nyquist to $11025/2 = 5512{,}5$ Hz. Najwyższa nuta modelu (MIDI 88, 1318,5 Hz) ma jeszcze 4 alikwoty poniżej Nyquista, a wyższe i tak są pomijane (w kodzie: alikwot powyżej $0{,}46 f_s$ = 5071,5 Hz jest zerowany). Jedna próbka trwa $1/11025 \approx 90{,}7$ μs.

(Dlaczego akurat 11 025 Hz? To 44 100 / 4. Ćwierć standardowego próbkowania płyt CD wystarcza, bo gitara nie niesie użytecznej informacji o wysokości powyżej kilku kHz, a obliczenia są 4× tańsze.)



Rysunek 1: Lewo: ton 2000 Hz próbkowany przy 11 025 Hz (pomarańczowe punkty) jest odtworzony wiernie. Prawo: ton 7000 Hz leży powyżej częstotliwości Nyquista; te same punkty pasują też do czerwonej fali 4025 Hz, więc po próbkowaniu nie da się ich odróżnić („zawijanie", *aliasing*). Dlatego przed zmianą próbkowania odfiltrowuje się wysokie tony.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Ile próbek ma 2 s nagrania przy 11 025 Hz? **Odp.:** $2 \cdot 11025 = 22\,050$.
2. Jaki najwyższy ton da się jeszcze poprawnie zapisać? **Odp.:** $f_s/2 = 5512{,}5$ Hz (Nyquist).
3. Co się stanie z tonem 7000 Hz? **Odp.:** zostanie zapisany jako „duch" 4025 Hz ($11025 - 7000$), bo leży powyżej Nyquista (rys. 16).

Najczęstszy błąd: uważanie, że „więcej próbek na sekundę" oznacza „wyższy dźwięk". Częstotliwość próbkowania to szybkość *pomiaru*, a częstotliwość tonu to szybkość *drgań*.

Co zapamiętać: Dźwięk to ciąg próbek; ton wyższy niż połowa częstotliwości próbkowania jest nie do odtworzenia.

M2. Wysokość dźwięku: Hz, półtony, centy, MIDI

Po co nam to? Model „myśli" w półtonach i binach, a nie w hercach. Ten rozdział uczy przeliczać jedno na drugie.

Intuicja. Ucho słyszy wysokość **logarytmicznie**: podwojenie częstotliwości to zawsze „oktawa wyżej", niezależnie od tego, czy z 100 Hz na 200 Hz, czy z 1000 na 2000. Dlatego muzycy dzielą oktawę na 12 równych (w sensie ilorazu) półtonów.

Wzór (MIDI → Hz). Numer nuty MIDI m (np. 69 = A4 = 440 Hz) ma częstotliwość

$$f(m) = 440 \cdot 2^{\frac{m-69}{12}} \text{ Hz}.$$

Każdy półton w górę mnoży częstotliwość przez $2^{1/12} \approx 1{,}0595$. Odwrotnie: $m = 69 + 12\log_2(f/440)$.

Centy. Półton dzielimy na 100 centów. Odstęp między częstotliwościami f_1, f_2 w centach to

$$c = 1200 \log_2 \frac{f_2}{f_1}.$$

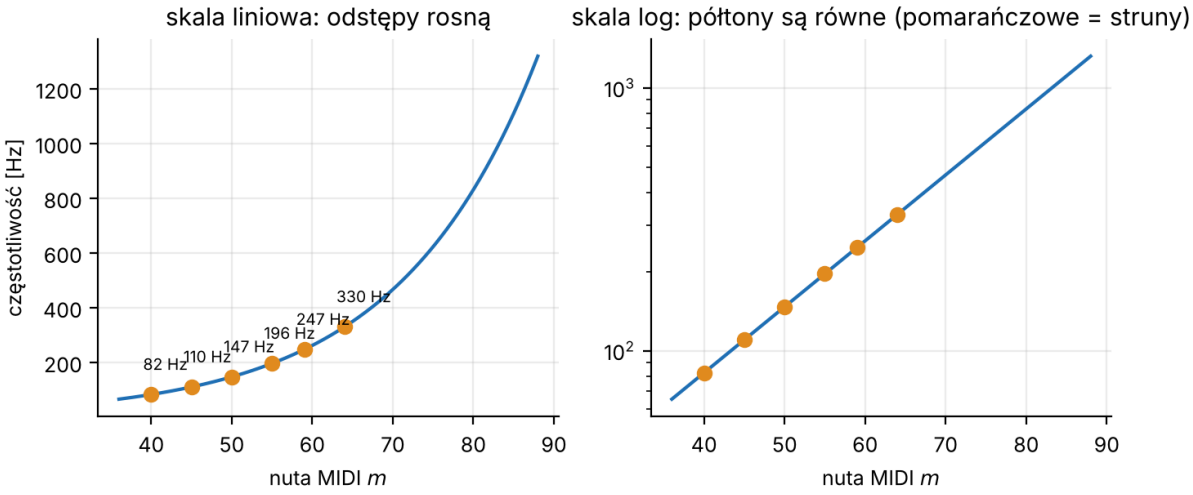
(Oktawa $= 1200$ ¢, półton $= 100$ ¢, bo $1200 \log_2 2 = 1200$ i $1200 \log_2 2^{1/12} = 100$.)

Przykłady.

- Najniższa struna gitary E2: $m = 40$, $f = 440 \cdot 2^{(40-69)/12} = 82{,}41$ Hz. Struna E4 ($m = 64$): 329,63 Hz.
- Strój gitary w kodzie: `tuning = (40, 45, 50, 55, 59, 64)` czyli E2 A2 D3 G3 B3 E4.
- Odstęp półtonu przy E2 to tylko $82{,}41 \cdot (2^{1/12} - 1) = 4{,}9$ Hz. To ważne: wąskie odstępy w niskich tonach wymagają **długiego okna** analizy (zob. M4).
- **Biny 1/3 półtonu.** Model dzieli każdy półton na 3 biny po $100/3 \approx 33{,}3$ ¢. Bin p ma wysokość MIDI

$$m(p) = 36 + \frac{p-1}{3}, \qquad p = 0, \ldots, 158.$$

Zatem bin $p = 3s+1$ leży **dokładnie** na półtonie $36+s$, a biny $3s$ i $3s+2$ to „odrobinę za nisko” i „odrobinę za wysoko” (±33 ¢). Dzięki temu gitara rozstrojona o kilkanaście centów nadal trafia w którąś z trzech komórek.



Rysunek 2: Częstotliwość nuty rośnie wykładniczo z numerem MIDI (lewy panel), ale na osi logarytmicznej (prawy panel) półtony są równo oddalone. Pomarańczowe punkty to puste struny gitary. Dlatego model liczy wszystko „po półtonach”, a nie po hercach.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Jaka jest częstotliwość nuty MIDI 57? **Odp.:** $440 \cdot 2^{(57-69)/12} = 440 \cdot 2^{-1} = 220$ Hz (A3, oktawa niżej niż 440 Hz).
2. Ile centów dzieli 220 Hz i 440 Hz? **Odp.:** $1200 \log_2 2 = 1200$ (jedna oktawa).
3. Który bin odpowiada nucie MIDI 45 (A2, struna 5 pusta)? **Odp.:** $p = 3 \cdot (45 - 36) + 1 = 28$.

Najczęstszy błąd: myślenie, że półton to stała liczba herców. Półton to stały *iloraz*

($\approx 1{,}0595$), więc przy niskich dźwiękach odstęp w Hz jest mały, a przy wysokich duży.

Co zapamiętać: Wysokość jest logarytmiczna: równa różnica w półtonach to równy iloraz częstotliwości.

M3. Alikwoty (harmoniczne)

Po co nam to? Właśnie alikwoty są powodem, dla którego rozpoznawanie nut jest trudne (błąd oktawowy) i jednocześnie powodem, dla którego jest wykonalne (nuta ma charakterystyczny „odcisk palca"). Blok harmoniczny sieci opiera się na tym rozdziale.

Intuicja. Szarpnięta struna nie drga tylko jednym tonem. Drga całą rodziną: z częstotliwością podstawową f_0 (to ją słyszymy jako wysokość nuty) i z jej **wielokrotnościami** — alikwotami, które nadają barwę. Dlatego nie wystarczy szukać piku widma w f_0: pik w $2f_0$ może być zarówno drugim alikwotem niższej nuty, jak i podstawą wyższej (**błąd oktawowy**, największy problem transkrypcji).

Wzór.

$$f_h = h \cdot f_0, \qquad h = 1, 2, 3, \ldots$$

W półtonach h-ty alikwot leży $12 \log_2 h$ półtonów nad podstawą, a w binach 1/3 półtonu:

$$k_h = 36 \log_2 h \quad \text{binów.}$$

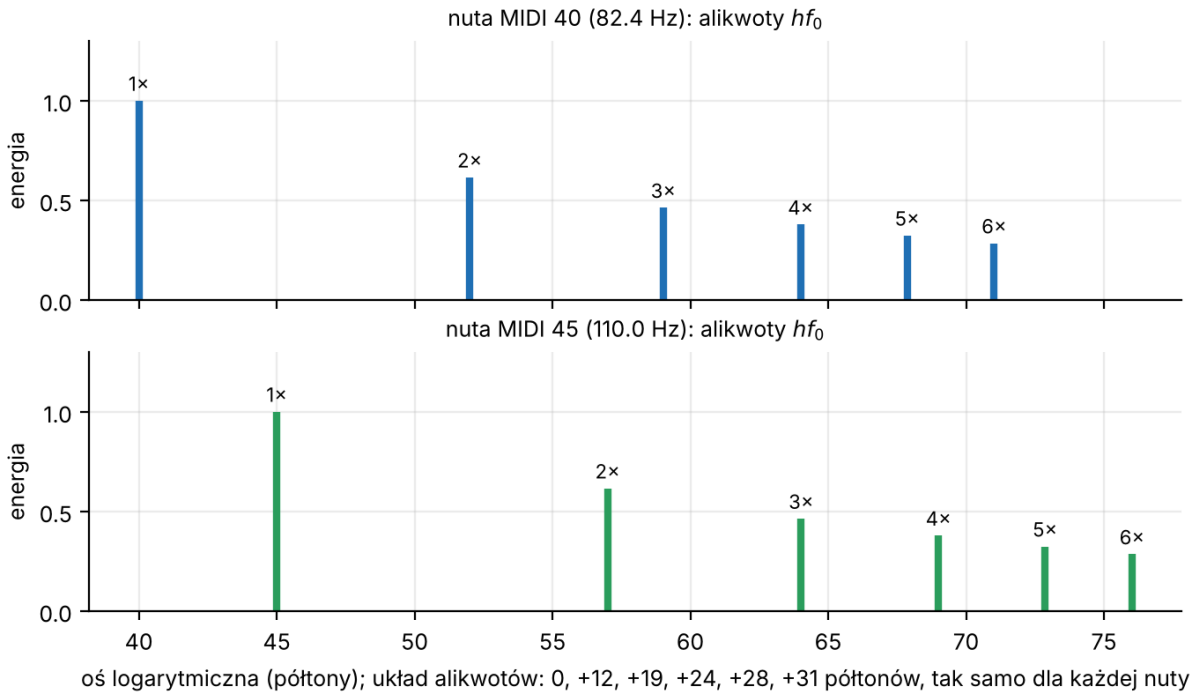
Przykład. Obliczone i zaokrąglone przesunięcia, których używa sieć (`harm_offsets`):

alikwot h	0,5	1	2	3	4	5	6
$36 \log_2 h$	−36	0	36	57,06	72	83,59	93,06
przesunięcie k_h (biny)	−36	0	36	57	72	84	93

Oktawa wyżej ($h = 2$) to dokładnie 12 półtonów $= 36$ binów; kwinta nad oktawą ($h = 3$) to $12 + 7 = 19$ półtonów ≈ 57 binów (zaokrąglenie, bo $12 \log_2 3 = 19{,}02$). Wartość $h = 0{,}5$ to **subharmoniczna**, czyli oktawa niżej: pozwala sieci zauważyć, że „to, co widzę, jest alikwotem czegoś niższego".

Kluczowa obserwacja. Na osi **logarytmicznej** (biny co 1/3 półtonu) układ alikwotów jest taki sam dla każdej nuty: zawsze −36, 0, +36, +57, … binów od podstawy. To pozwala użyć jednego, wspólnego „detektora wzorca" dla wszystkich wysokości (zob. M6, blok harmoniczny).

Przykład krok po kroku (nuta E2, struna 6 pusta). Półton $s = 40 - 36 = 4$, więc bin $p = 3 \cdot 4 + 1 = 13$. Blok harmoniczny czyta z tego binu siedem miejsc: $p + k$ dla $k = -36, 0, 36, 57, 72, 84, 93$, czyli biny -23 (poza zakresem, wstawia zera), 13, 49, 70, 85, 97, 106. Zamieniamy je z powrotem na nuty ze wzoru $m(p) = 36 + (p-1)/3$: bin 13 → 40 (E2), 49 → 52 (E3), 70 → 59 (B3), 85 → 64 (E4), 97 → 68 (G♯4), 106 → 71 (B4). To dokładnie 1., 2., 3., 4., 5. i 6. alikwot E2. Jeśli w tych siedmiu miejscach wszędzie jest energia, kandydat „E2" jest bardzo prawdopodobny. Jeśli natomiast w binie 13 (podstawa) nic nie ma, a w 49, 85 i 106 jest energia, to raczej gra E3 (jego alikwoty to właśnie 49, 85, 106…), a „E2" byłoby błędem oktawowym. Blok harmoniczny widzi oba te miejsca naraz, więc może to rozróżnić.



Rysunek 3: Dwie różne nuty (MIDI 40 i 45) na osi logarytmicznej. Prążki to alikwoty $1\times, 2\times, \ldots, 6\times$. Wzór prążków jest identyczny, tylko przesunięty. Dzięki temu jeden filtr rozpoznaje „nutę" w każdej wysokości.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Jakie są częstotliwości pierwszych trzech alikwotów A2 (110 Hz)? **Odp.:** 110, 220, 330 Hz. Trzeci (330 Hz) to prawie dokładnie E4 (329,63 Hz), czyli $12 + 7 = 19$ półtonów wyżej.
2. Ile binów to 2. oktawa (4. alikwot)? **Odp.:** $36 \log_2 4 = 72$ biny $= 24$ półtony.
3. Dlaczego wzór alikwotów jest taki sam dla każdej nuty na osi logarytmicznej? **Odp.:** bo mnożenie częstotliwości (każdy alikwot $= h \cdot f_0$) to na osi logarytmicznej *dodawanie* stałej ($\log h$).

Najczęstszy błąd: mylenie „h-tego alikwotu" z „h-tą nutą skali". Alikwot h leży $12 \log_2 h$ półtonów nad podstawą, więc 3. alikwot to nie trzecia nuta, tylko kwinta nad oktawą.

Co zapamiętać: Nuta = podstawa + jej wielokrotności. Na osi logarytmicznej układ tych wielokrotności jest zawsze taki sam, więc można go wykryć jednym wspólnym filtrem.

M4. Transformata Fouriera: z drgań do widma

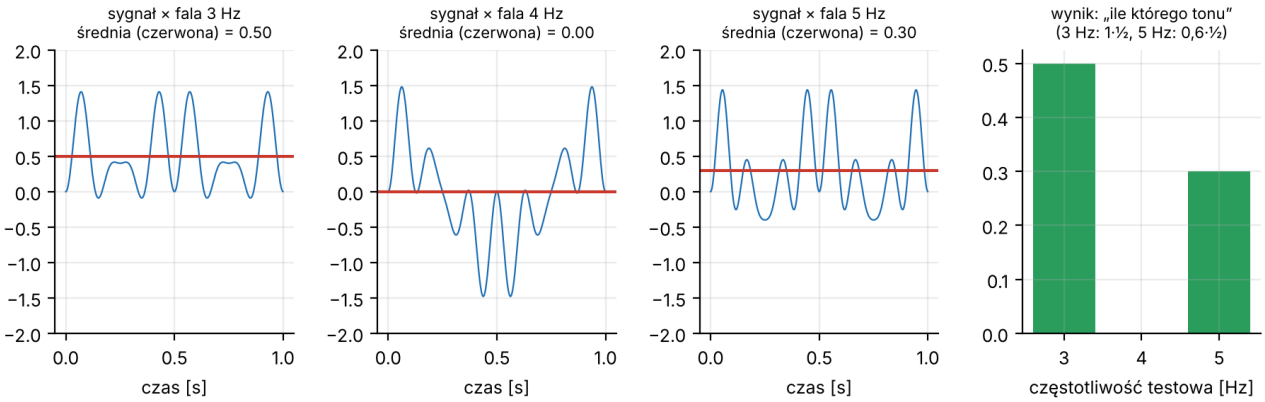
Po co nam to? Z surowych próbek nie widać, które tony grają. Transformata Fouriera „rozbija" dźwięk na tony, a wybór długości okna decyduje, czy rozróżnimy dwa sąsiednie półtony.

Intuicja. Transformata Fouriera sprawdza, **ile** czystego tonu o każdej częstotliwości siedzi w krótkim fragmencie dźwięku — jak rozdzielenie akordu na pojedyncze struny.

Wzór (DFT). Dla ramki N próbek $x[0..N-1]$:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-2\pi i\, kn/N}, \qquad k = 0, \dots, N-1.$$

Wzór Eulera $e^{-i\phi} = \cos\phi - i\sin\phi$ pokazuje, co się dzieje: mnożymy sygnał przez falę cosinusową i sinusową o „numerze” k i sumujemy. Jeśli sygnał zawiera ton o tej częstotliwości, wyrazy dodają się w fazie i suma jest duża; w przeciwnym razie plusy i minusy się znoszą. **Moduł** $|X[k]|$ mówi, *jak silny* jest ton k (faza nas tu nie interesuje).



Rysunek 4: Sedno DFT na prostym przykładzie: sygnał to suma tonu 3 Hz (amplituda 1) i 5 Hz (amplituda 0,6). Mnożymy go przez falę testową i liczymy średnią (czerwona linia). Dla fali 3 Hz średnia to 0,5, dla 5 Hz 0,3, a dla 4 Hz — zero, bo takiego tonu w sygnale nie ma (plusy i minusy się znoszą). Prawy panel to „widmo” otrzymane z tych trzech średnich (po przemnożeniu przez 2 daje amplitudy 1 i 0,6). W prawdziwej DFT robi się to dla każdej częstotliwości naraz, a dodatkowo z falą cosinusową, żeby uwzględnić fazę.

Biny k odpowiadają częstotliwościom kf_s/N. **Rozdzielczość** widma to

$$\Delta f = \frac{f_s}{N}.$$

Okno Hanna. Ucięcie sygnału do N próbek „odcina” go gwałtownie i rozmywa widmo (przeciek). Mnożymy więc ramkę przez gładkie okno

$$w[n] = \tfrac{1}{2}\left(1 - \cos\frac{2\pi n}{N}\right), \qquad n = 0, \dots, N-1,$$

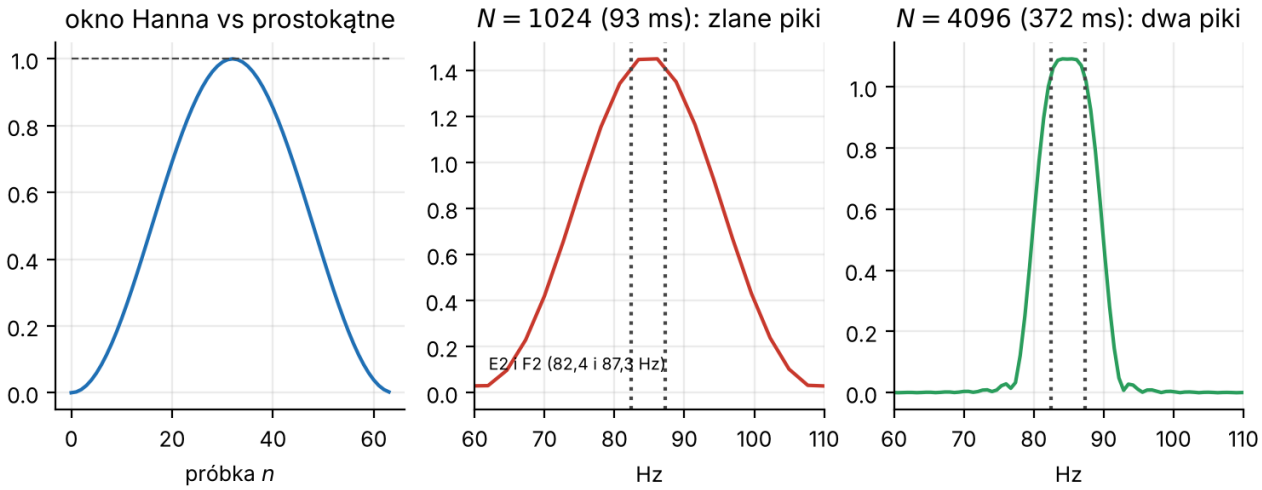
które zaczyna i kończy się zerem (kod: `np.hanning(N+1)[:N]`, tzw. okno okresowe). Z oknem Hanna czysty ton o amplitudzie A daje $|X| \approx A\,N/4$ (średnia okna to $\frac{1}{2}$, a widmo rzeczywistego sygnału dzieli się na dwie połówki ±częstotliwość, razem $\frac{1}{2}\cdot\frac{1}{2}$). Dlatego kod mnoży wynik przez $4/N$, aby odczytać **amplitudę** A **wprost**.

Dopełnianie zerami (zero-padding). Gdy do ramki dołożymy zera do długości $M = z \cdot N$ ($z = 4$ w modelu), widmo staje się „gęściej próbkowane” (biny co f_s/M) — nie zyskujemy nowej informacji, ale krzywa jest gładsza i łatwiej odczytać położenie piku.

Kompromis czas–częstotliwość. Długie okno = dobra rozdzielczość częstotliwości, zła w czasie; krótkie okno odwrotnie. Gen 6 używa obu:

okno	N	czas trwania N/f_s	$\Delta f = f_s/N$	biny po zero-padding ($M = 4N$)
długie	4096	371,5 ms	2,69 Hz	co 0,673 Hz
krótkie	1024	92,9 ms	10,77 Hz	co 2,69 Hz

Półton przy E2 to 4,9 Hz, więc tylko okno długie (2,69 Hz) potrafi go rozdzielić; dlatego jest potrzebne. Krótkie okno lepiej „widzi” moment szarpnięcia.



Rysunek 5: Lewo: okno Hanna wygasza brzegi ramki (linia przerywana: okno prostokątne). Środek i prawo: dwie nuty, E2 (82,4 Hz) i F2 (87,3 Hz), oddalone o półton. Krótkie okno (93 ms) zlewa je w jeden garb, długie (372 ms) rozdziela w dwa piki. Dlatego potrzebne są oba okna.

Krok (hop). Ramki przesuwamy o hop $= 256$ próbek: $256/11025 = 23{,}2$ ms. Klatka t ma środek w próbce $t \cdot 256 + N_L/2$, czyli w czasie

$$\tau_t = \frac{256\,t + 2048}{11025}\ \text{s}.$$

Obie ramki (długa i krótka) w klatce t mają **ten sam środek**.

Przykład wymiarów. Nagranie 23 s ($\approx 253\,575$ próbek) daje $T = (253575 - 4096)/256 + 1 = 975$ klatek.

Przykład: czas klatki. Klatka $t = 35$ ma środek w $\tau_{35} = (256 \cdot 35 + 2048)/11025 = (8960 + 2048)/11025 = 0{,}9984$ s, czyli niemal dokładnie w 1,0 s nagrania.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Rozdzielczość FFT dla $N = 2048$, $f_s = 11025$? **Odp.:** $11025/2048 = 5{,}38$ Hz.
2. Czy takie okno rozdzieli E2 (82,41 Hz) i F2 (87,31 Hz)? **Odp.:** nie: odstęp to 4,9 Hz, a rozdzielczość 5,38 Hz jest gorsza (rys. 3).

3. Co daje dopełnianie zerami? **Odp.:** gładszy wykres widma, ale *nie* lepszą rozdzielczość (nowej informacji nie ma).

Najczęstszy błąd: wiara, że dopełnienie zerami poprawia rozdzielczość. Poprawia tylko interpolację, rozdzielczość zależy wyłącznie od długości *prawdziwego* sygnału w oknie.

Co zapamiętać: Długie okno widzi częstotliwości dokładnie, ale czas niedokładnie; krótkie odwrotnie. Dlatego HCQT używa obu.

M5. HCQT: widmo „ułożone" harmonicznie

Po co nam to? To jest dosłownie to, co dostaje sieć. Wszystko, co dalej, działa na tym tensorze, więc warto wiedzieć, skąd się bierze każda liczba.

Intuicja. Zwykłe widmo ma biny liniowe (co f_s/M Hz), a my chcemy biny logarytmiczne (co 1/3 półtonu) i dodatkowo — dla każdej kandydującej wysokości — informację, ile energii jest w jej 1×, 2×, 3×… wielokrotności. To właśnie **HCQT** (*Harmonic Constant-Q Transform*, Bittner i in. 2017, tu w odmianie liczonej z FFT).

Wzór. Dla kanału c z listą alikwotów h_c, klatki t i binu p (wysokość $f(p) = 440 \cdot 2^{(m(p)-69)/12}$):

$$A_c(t,p) = \frac{4}{N_c} \max_{k \in [a,\,b]} \left|X_t^{(c)}[k]\right|, \qquad a = \left\lfloor \frac{h_c f(p)\, 2^{-1/72}}{f_s/M_c} \right\rfloor, \quad b = \left\lceil \frac{h_c f(p)\, 2^{+1/72}}{f_s/M_c} \right\rceil.$$

Czyli: patrzymy na częstotliwość $h_c f(p)$ i bierzemy **maksimum** widma w przedziale $\pm 1/72$ oktawy $= \pm 1/6$ półtonu ($\pm 16{,}7$ ¢) wokół niej. M_c to długość FFT po zero-padding. Jeśli $h_c f(p) > 0{,}46 f_s$, wynik to 0.

Dziewięć kanałów ($C = 9$): sześć z okna długiego z alikwotami $h \in \{0{,}5; 1; 2; 3; 4; 5\}$ i trzy z okna krótkiego z $h \in \{1; 2; 3\}$. Kanał „$h = 2$" w binie p mówi więc: „ile jest energii w oktawie nad p".

Przykład liczbowy (E2). Bin dokładnie na $m = 40$: $f = 82{,}41$ Hz. Przedział $\pm 1/72$ oktawy to 81,62–83,20 Hz. Przy biniach co 0,673 Hz (okno długie, $M = 16384$) wypada $a = \lfloor 121{,}3 \rfloor = 121$, $b = \lceil 123{,}6 \rceil = 124$, czyli maksimum z 4 binów FFT. Dla $h = 2$ (164,81 Hz): biny 242–248.

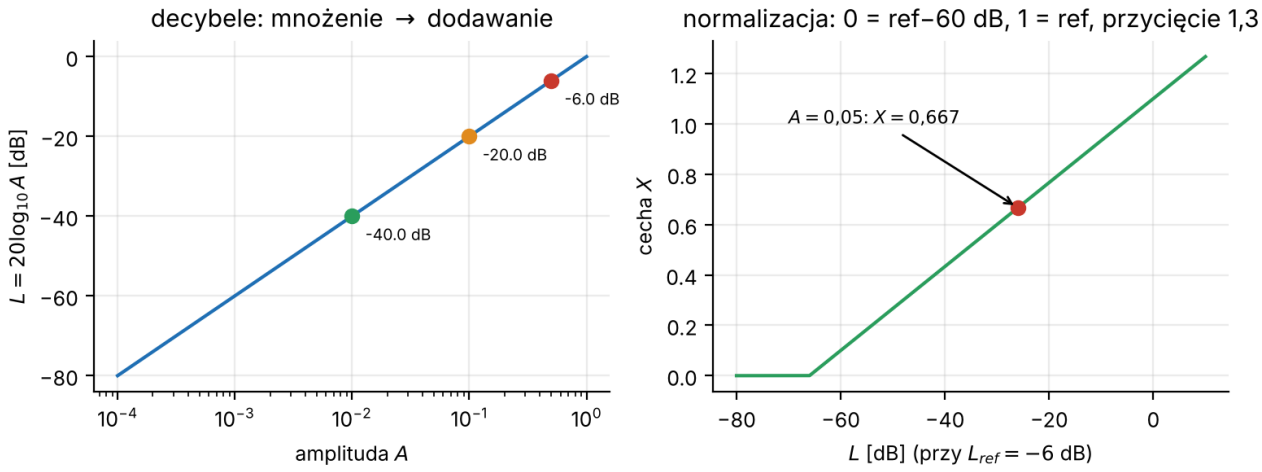
Decybele i normalizacja

Decybele. Ucho i dynamika nagrań są logarytmiczne, więc amplitudę A zamieniamy na decybele:

$$L = 20 \log_{10}(A + 10^{-9}) \text{ dB}.$$

(Człon 10^{-9} chroni przed $\log 0$.) Spadek amplitudy o połowę to $20 \log_{10} 0{,}5 = -6{,}02$ dB; dziesięciokrotny spadek to -20 dB.

Normalizacja względem nagrania. Dla każdej grupy kanałów (długie osobno, krótkie osobno) liczymy L_{ref} **= 99,5. percentyl** wartości L w całym nagraniu (percentyl odporny na pojedyncze skoki; do liczenia używa się co 4. klatki). Wartość końcowa:



Rysunek 6: Lewo: decybele zamieniają mnożenie amplitud na dodawanie: połowa amplitudy to -6 dB, jedna dziesiąta -20 dB, jedna setna -40 dB. Prawo: normalizacja $X = (L - L_{\text{ref}})/60 + 1$ (tu dla $L_{\text{ref}} = -6$ dB) i przycięcie do $[0;\ 1{,}3]$. Czerwony punkt to przykład z tekstu ($A = 0{,}05$).

$$\boxed{X(t, c, p) = \text{clip}\Big(\frac{L - L_{\text{ref}}}{60} + 1,\ 0,\ 1{,}3\Big)}$$

Interpretacja: 1,0 = głośność równa „prawie najgłośniejszemu” miejscu nagrania, 0 = 60 dB cichsze (cisza), a górne przycięcie 1,3 odpowiada $+18$ dB. Dzięki temu sieć jest **niewrażliwa na ustawienie głośności nagrania**.

Wynik: tensor $X \in \mathbb{R}^{T \times 9 \times 159}$ (kod: `features.py:hcqt`).

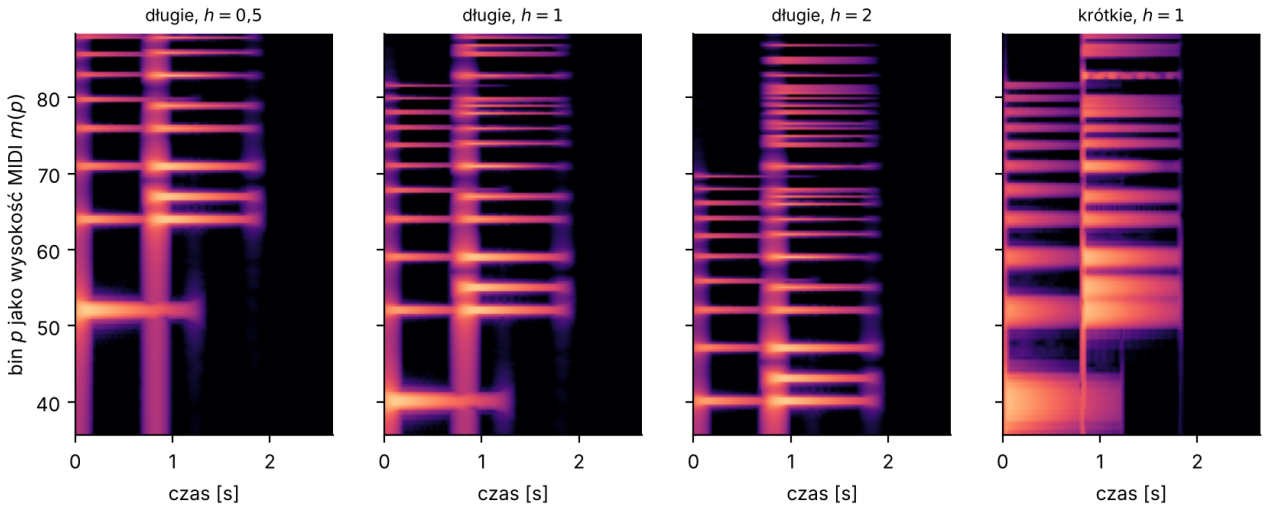
Przykład liczbowy normalizacji. Niech w pewnym punkcie amplituda wynosi $A = 0{,}05$, a 99,5. percentyl nagrania wynosi $L_{\text{ref}} = -6$ dB. Wtedy $L = 20\log_{10} 0{,}05 = -26{,}0$ dB, więc $X = (-26 + 6)/60 + 1 = 0{,}667$. Gdyby A było dwa razy większe, L wzrosłoby o 6 dB, a X o $6/60 = 0{,}1$. Punkt 60 dB poniżej odniesienia ($L = -66$ dB) daje $X = 0$.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Dla $A = 0{,}1$ i $L_{\text{ref}} = -6$ dB ile wynosi X? **Odp.:** $L = -20$ dB, $X = (-20 + 6)/60 + 1 = 0{,}767$.
2. Co oznacza $X = 0$? **Odp.:** poziom co najmniej 60 dB poniżej odniesienia, czyli praktycznie cisza.
3. Dlaczego normalizujemy względem percentyla nagrania? **Odp.:** by sieć nie zależała od tego, jak głośno nagrano (jedno nagranie ma ten sam wynik po wzmocnieniu o 10 dB).

Najczęstszy błąd: branie maksimum widma w pojedynczym binie FFT. Model bierze maksimum z *przedziału* $\pm 1/6$ półtonu, żeby rozstrojenie nie wyrzuciło piku poza komórkę.

Co zapamiętać: HCQT to widmo w skali logarytmicznej, z kanałem na każdy alikwot, w decybelach i znormalizowane do głośności nagrania.



Rysunek 7: Prawdziwe wyjście `features.py:hcqt` dla syntetycznego sygnału: nuta E2 (od 0,2 s) i trzy nuty akordu (E3, G3, B3, od 1,0 s). Jasność to wartość X. Kanał h w binie p pokazuje energię przy częstotliwości $h \cdot f(p)$. Dlatego nuta E2 zapala się w binie E2 w każdym kanale z $h \geq 1$ (jej alikwoty leżą w hf_0), a dodatkowo: nuta E3 zapala się w kanale $h = 2$ w binie E2 (bo $2f(\mathrm{E2}) = f(\mathrm{E3})$), a w kanale $h = 0{,}5$ nuta E2 zapala się w binie E3 (bo $0{,}5\, f(\mathrm{E3}) = f(\mathrm{E2})$). Tak powstają „fałszywi kandydaci", które sieć musi odrzucić. Kanał krótki (po prawej) wyraźnie pokazuje moment szarpnięcia (pionowa kreska w 1,0 s).

M6. Konwolucja i jej odmiany

> **Po co nam to?** Konwolucje to „oczy" sieci: małe filtry przesuwane po widmie, które uczą się rozpoznawać wzory (piki, szeregi alikwotów, początki nut).

Intuicja. Konwolucja to „przesuwane okienko": mały zestaw wag (filtr, *kernel*) przesuwa się po danych i w każdym miejscu liczy ważoną sumę sąsiadów. Ten sam filtr działa wszędzie, więc jeśli nauczył się rozpoznawać „pik", rozpozna go w każdej wysokości i w każdej chwili. Mało parametrów, wbudowana niezmienniczość względem przesunięcia.

Wzór (2D, czas × częstotliwość). Wejście x o C_{in} kanałach, filtr o rozmiarze $K_t \times K_p$, wyjście y o C_{out} kanałach:

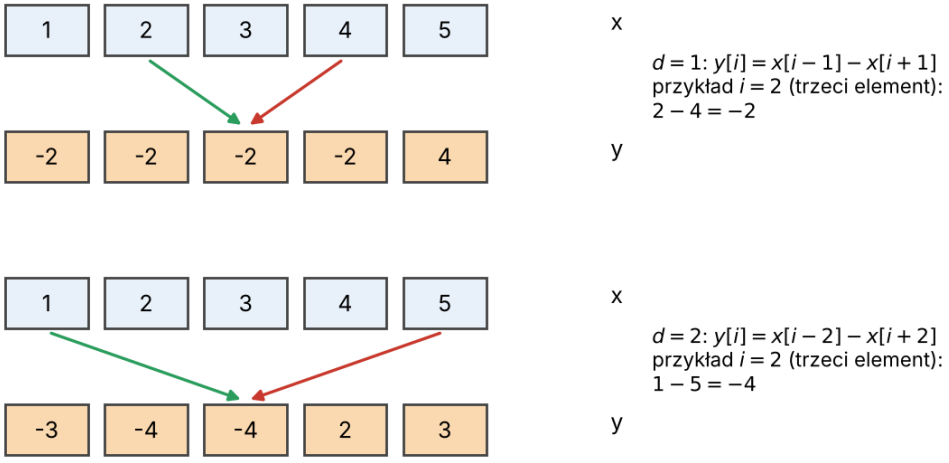
$$y_o[t,p] = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_i \sum_j W_{o,c}[i,j]\; x_c[t + d\,i,\; p + j]$$

(d — **dylatacja**, patrz niżej; indeksy i, j biegną symetrycznie wokół zera, np. $-1, 0, 1$.) Liczba parametrów jednej warstwy: $C_{\text{out}} \cdot C_{\text{in}} \cdot K_t K_p + C_{\text{out}}$ (ostatni składnik to obciążenia b_o).

Przykład. Trzon sieci: $C_{\text{in}} = 9$, $C_{\text{out}} = H = 48$, filtr 3×7: $9 \cdot 48 \cdot 21 + 48 = 9120$ parametrów. Filtr obejmuje 3 klatki × 7 binów (7 binów ≈ 2,3 półtonu).

Przykład ręczny (1D, żeby było widać każdy krok). Dane $x = (1, 2, 3, 4, 5)$, filtr o trzech wagach $W = (1, 0, -1)$ (indeksy $-1, 0, 1$), poza krawędziami zera. Wzór daje $y[i] = x[i-1] - x[i+1]$, czyli $y = (0-2,\ 1-3,\ 2-4,\ 3-5,\ 4-0) = (-2, -2, -2, -2, 4)$. Z dylatacją $d = 2$ ten sam filtr patrzy dalej: $y[i] = x[i-2] - x[i+2]$, więc $y = (0-3,\ 0-4,\ 1-5,\ 2-0,\ 3-0) = (-3, -4, -4, 2, 3)$. Liczba wag się nie zmieniła (3), a pole widzenia urosło. (W bibliotekach

uczenia głębokiego „konwolucja” liczy dokładnie taką sumę bez odwracania filtra, i tu też.)



Rysunek 8: Ten sam przykład jako rysunek: element wyjścia $y[2]$ powstaje z dwóch elementów wejścia leżących w odległości d po obu stronach: zielona strzałka to $x[i-d]$ (z plusem), czerwona to $x[i+d]$ (z minusem). Większe d = ten sam filtr patrzy dalej, bez dokładania parametrów.

Warianty użyte w modelu:

1. **Konwolucja 1×1 (punktowa)** — w każdym punkcie miesza tylko kanały: $y_o[t,p] = b_o + \sum_c W_{o,c}\, x_c[t,p]$. To zwykła warstwa liniowa działająca niezależnie w każdym (t,p).
2. **Konwolucja głębokościowa (*depthwise*)** — każdy kanał ma własny filtr i kanały się nie mieszają (groups $= H$): $y_c[t,p] = b_c + \sum_j W_c[j]\, x_c[t,p+j]$. Parametrów $H \cdot K + H$ zamiast H^2K. W modelu: filtr 1×5, czyli $48 \cdot 5 + 48 = 288$ parametrów; służy do uwzględnienia sąsiednich binów (rozstrojenie, nieharmoniczność strun).
3. **Konwolucja z dylatacją** d — filtr „rozciągnięty”: pobiera próbki co d pozycji, nie zwiększając liczby parametrów. Przy trzech blokach $d = 1, 2, 4$ pole widzenia w czasie rośnie wykładniczo.
4. **Krok (*stride*) 3** — wynik liczymy co 3. pozycję. Używamy go raz, by zejść z binów 1/3 półtonu do półtonów: $159 \to 53$.

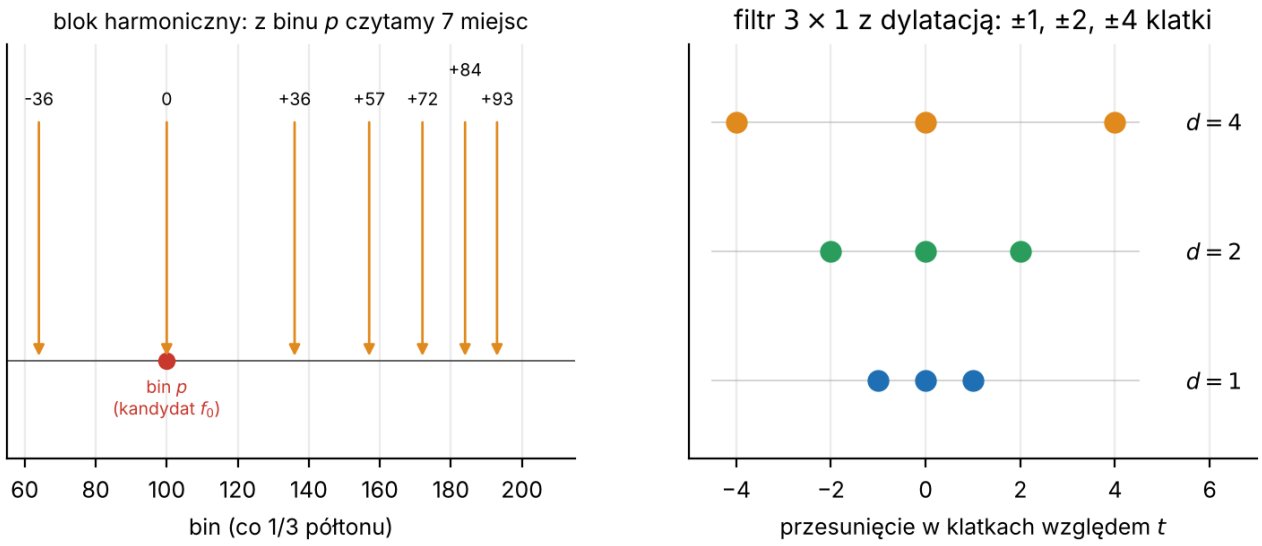
Pole recepcyjne (ile wejściowych klatek wpływa na jedną klatkę wyjścia). Filtr czasowy 3 z dylatacją d dodaje d klatek z każdej strony. Trzy bloki $d = 1, 2, 4$ dają $1 + 2 + 4 = 7$ klatek na stronę, zob. M9 i rozdz. 5.6.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Ile parametrów ma konwolucja 1×1 z 336 do 48 kanałów? **Odp.:** $336 \cdot 48 + 48 = 16\,176$.
2. Na jakie klatki patrzy filtr 3×1 z dylatacją 4 w klatce t? **Odp.:** $t-4,\ t,\ t+4$.
3. Ile klatek z każdej strony widzą trzy bloki z dylatacjami 1, 2, 4? **Odp.:** $1+2+4 = 7$.

Najczęstszy błąd: mylenie wymiaru „kanały” z wymiarem „czas”. Kanał to *rodzaj cechy* w danym punkcie (np. alikwot), a nie kolejna chwila.

Co zapamiętać: Konwolucja = ten sam mały filtr wszędzie. Dylatacja zwiększa zasięg bez dokładania wag, a konwolucja 1×1 tylko miesza kanały.



Rysunek 9: Lewo: bin p (czerwony) i siedem miejsc, z których czyta go blok harmoniczny (liczby to przesunięcia w binach). Prawo: filtr czasowy 3×1 z dylatacją $d = 1, 2, 4$ pobiera klatki $t, t\pm d$; trzy bloki z rosnącą dylatacją widzą coraz szerszy czas bez dokładania parametrów.

M7. Funkcje nieliniowe: sigmoid, softmax, GELU

> **Po co nam to?** Sieć wypluwa dowolne liczby, a my chcemy prawdopodobieństw (od 0 do 1). Sigmoid i softmax robią to przekształcenie, a GELU daje sieci zdolność uczenia się krzywych, nie tylko prostych.

Bez nieliniowości głęboka sieć byłaby jedną wielką mnożarką macierzy. Dlatego po warstwach liniowych stosuje się nieliniowe „zgięcia”.

Sigmoid (zamienia dowolną liczbę — *logit* — na prawdopodobieństwo):

$$\sigma(z) = \frac{1}{1+e^{-z}} \in (0,1).$$

$\sigma(0) = 0{,}5$; $\sigma(-3) = 0{,}047$; $\sigma(2) = 0{,}88$. Odwrotność to *logit* $z = \ln\frac{p}{1-p}$.

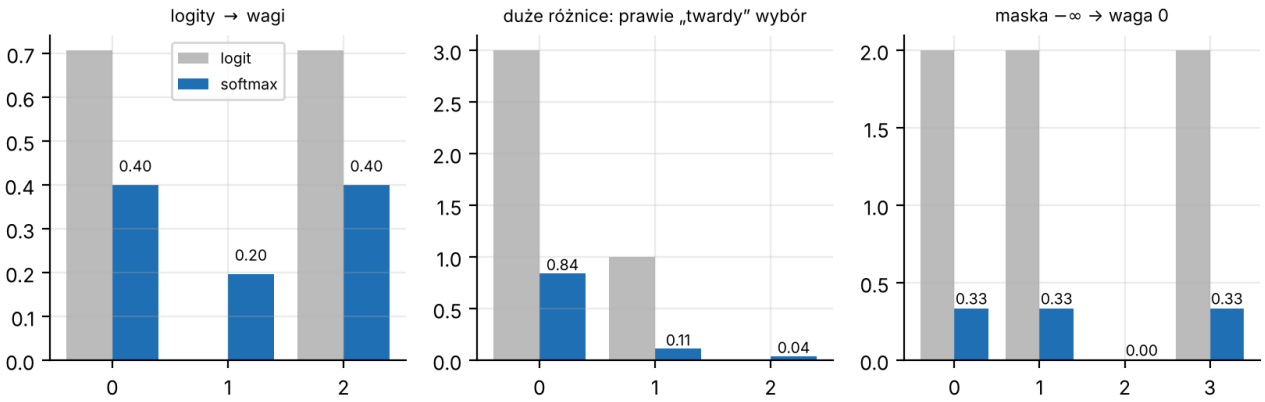
Softmax (zamienia wektor $z_1, \ldots, z_n$ na rozkład prawdopodobieństwa — nieujemne liczby sumujące się do 1):

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Przykład: $z = (0{,}707;\ 0;\ 0{,}707) \Rightarrow e^z = (2{,}03;\ 1;\ 2{,}03)$, suma $5{,}06$, wynik $(0{,}401;\ 0{,}198;\ 0{,}401)$. Wartość $-\infty$ daje wagę 0 (tak „wyłączamy” niedozwolone pozycje).

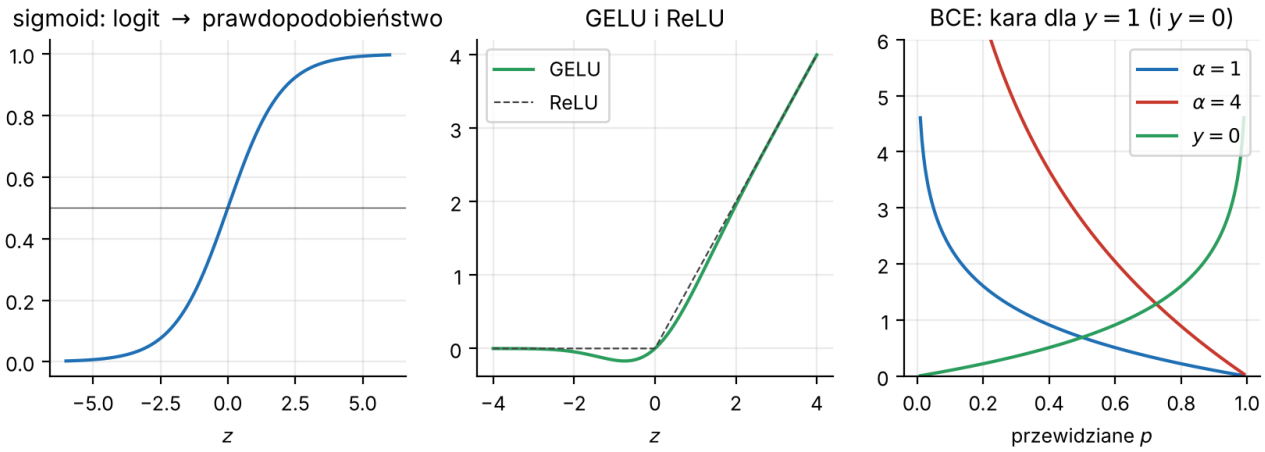
GELU (gładka wersja „ReLU”; przepuszcza duże dodatnie wartości, tłumi ujemne):

$$\mathrm{GELU}(z) = z\cdot\Phi(z) = z\cdot\tfrac{1}{2}\Big(1+\mathrm{erf}\,\tfrac{z}{\sqrt{2}}\Big),$$



Rysunek 10: Trzy zachowania softmax: (1) wagi z przykładu liczbowego (0,40/0,20/0,40); (2) duże różnice logitów dają niemal „twardy" wybór największego; (3) wartość $-\infty$ (maska) daje wagę dokładnie 0, a reszta dzieli się po równo.

gdzie $\Phi(z)$ to dystrybuanta rozkładu normalnego (pole pod krzywą dzwonową od $-\infty$ do z; $\Phi(0) = 0{,}5$, $\Phi(1) = 0{,}84$), a erf to funkcja błędu (M0), z której się ją liczy. $\mathrm{GELU}(-1) = -0{,}159$, $\mathrm{GELU}(0) = 0$, $\mathrm{GELU}(1) = 0{,}841$, $\mathrm{GELU}(2) = 1{,}954$.



Rysunek 11: Sigmoid (lewo), GELU z porównaniem do ReLU (środek) oraz kara BCE (prawo): dla $y = 1$ kara $-\alpha \ln p$ rośnie bardzo szybko, gdy $p \to 0$; czerwona krzywa ($\alpha = 4$) leży 4 razy wyżej niż niebieska, więc przeoczenie początku nuty boli mocniej niż fałszywy alarm.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Ile wynosi $\sigma(0)$? **Odp.:** 0,5.
2. Ile wynosi $\mathrm{softmax}(0, 0)$? **Odp.:** $(0{,}5;\ 0{,}5)$ (równe logity dają równe wagi).
3. $\mathrm{softmax}(\ln 1,\ \ln 3) =$? **Odp.:** $e^{\ln 1} = 1$, $e^{\ln 3} = 3$, suma 4, więc $(0{,}25;\ 0{,}75)$.

Najczęstszy błąd: użycie softmax dla zdarzeń, które mogą zajść *razem* (kilka nut naraz). Wtedy każde z nich osobno dostaje sigmoid, a softmax jest dla wyboru jednej opcji z kilku (jak struna).

Co zapamiętać: Sigmoid = „czy tak?", softmax = „która z opcji?", GELU = „zgięcie" dające nieliniowość.

M8. Normalizacja warstw, połączenia resztkowe, dropout

Po co nam to? Głęboka sieć łatwo „wariuje”: liczby rosną, gradient znika. LayerNorm, skróty resztkowe i dropout to trzy sposoby, żeby uczenie było stabilne.

LayerNorm. Dla wektora cech $z \in \mathbb{R}^d$ (np. $d = 64$ kanałów w jednym punkcie):

$$\mathrm{LN}(z)_i = \gamma_i \frac{z_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i, \qquad \mu = \frac{1}{d}\sum_i z_i, \quad \sigma^2 = \frac{1}{d}\sum_i (z_i - \mu)^2.$$

Czyli: odejmij średnią, podziel przez odchylenie standardowe (wektor ma średnią 0 i wariancję 1), a potem pozwól sieci nauczyć się własnej skali γ_i i przesunięcia β_i ($2d$ parametrów). Stabilizuje uczenie: wartości nie rosną ani nie maleją w kolejnych warstwach.

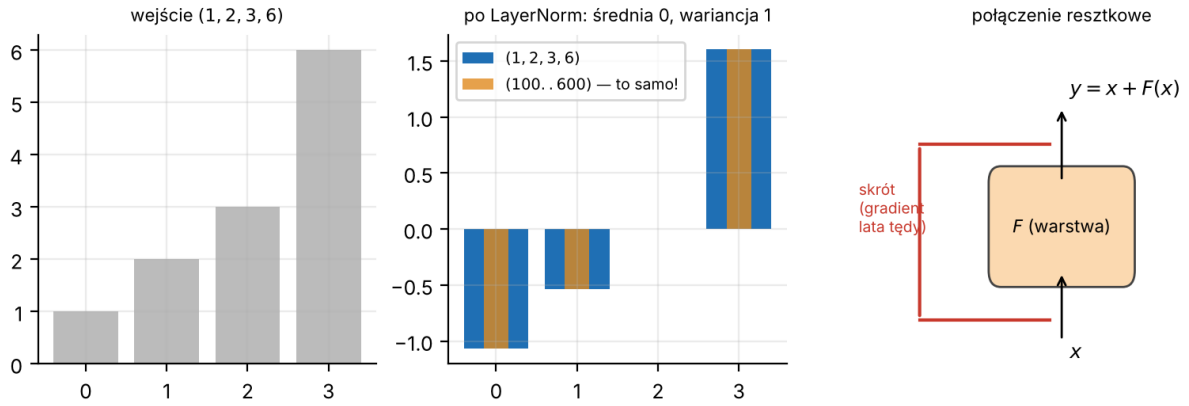
Przykład. $z = (1, 2, 3, 6)$: $\mu = 3$, $\sigma^2 = (4 + 1 + 0 + 9)/4 = 3{,}5$, $\sigma = 1{,}871$. Po normalizacji (dla $\gamma = 1, \beta = 0$): $(-1{,}069;\ -0{,}535;\ 0;\ 1{,}604)$. Średnia jest 0, a wariancja 1, niezależnie od tego, czy wektor był $(1, 2, 3, 6)$, czy $(100, 200, 300, 600)$.

Połączenie resztkowe (*residual*). Zamiast $y = F(x)$ liczymy

$$y = x + F(x).$$

Warstwa uczy się tylko **poprawki** do wejścia, a gradient może płynąć przez „skrót” x bez tłumienia — dzięki temu można układać wiele warstw. Wariant **pre-norm** stosowany w kontekście: $y = x + F(\mathrm{LN}(x))$.

Dropout ($p = 0{,}1$). Podczas uczenia każdy element jest z prawdopodobieństwem p zerowany, a pozostałe mnożone przez $1/(1-p)$ (żeby średnia się zgadzała). Zapobiega „wykuciu na pamięć”. Przy wnioskowaniu wyłączony.



Rysunek 12: Lewo i środek: LayerNorm „zapomina” skalę wektora: $(1, 2, 3, 6)$ i $(100, 200, 300, 600)$ dają ten sam wynik (słupki się pokrywają). Prawo: połączenie resztkowe $y = x + F(x)$: warstwa F dodaje poprawkę, a czerwony „skrót” pozwala gradientowi płynąć bez tłumienia.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Znormalizuj $(2, 4, 6)$ przez LayerNorm ($\gamma = 1, \beta = 0$). **Odp.:** $\mu = 4$, $\sigma = 1{,}633$, wynik $(-1{,}225;\ 0;\ 1{,}225)$.

2. Co oznacza $y = x + F(x)$? **Odp.:** warstwa F dodaje poprawkę do wejścia, zamiast je zastępować.
3. Czy dropout działa przy wnioskowaniu? **Odp.:** nie, tylko przy uczeniu.

Najczęstszy błąd: sądzenie, że LayerNorm uczy się „średniej z całego zbioru". Liczy się osobno dla każdego wektora, a uczone są tylko γ i β.

Co zapamiętać: Normalizacja stabilizuje liczby, skrót resztkowy ułatwia przepływ gradientu, dropout zapobiega wkuwaniu na pamięć.

M9. Uwaga (*attention*)

Po co nam to? Nuta nie jest zdarzeniem jednej klatki, więc sieć musi patrzeć na kontekst: wcześniejsze i późniejsze chwile (uwaga w czasie) oraz inne wysokości (uwaga między wysokościami). Uwaga to narzędzie, które wybiera, na co patrzeć.

Intuicja. Chcemy, żeby każdy element sekwencji (np. klatka w czasie) mógł **wybrać**, na które inne elementy patrzeć i je uśrednić. Robi to mechanizm „zapytanie–klucz–wartość": każdy element wystawia *zapytanie* (q: „czego szukam?"), *klucz* (k: „co oferuję?") i *wartość* (v: „co przekażę, jeśli mnie wybierzesz"). Im bardziej zapytanie pasuje do klucza (duży iloczyn skalarny), tym większą wagę ma wartość.

Wzór. Dla sekwencji n wektorów $x_1, \dots, x_n \in \mathbb{R}^D$:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i,$$

$$\boxed{y_i = \sum_j a_{ij}\, v_j, \qquad a_{ij} = \mathrm{softmax}_j\Big(\frac{q_i \cdot k_j}{\sqrt{d_h}} + b_{\,j-i}\Big)}$$

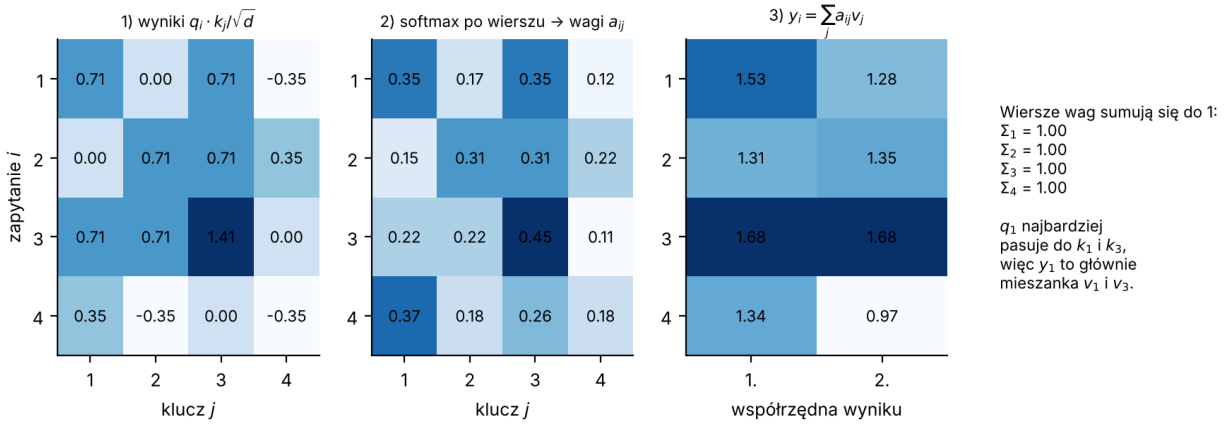
- $q_i \cdot k_j = \sum_l q_{il} k_{jl}$ — iloczyn skalarny (miara podobieństwa);
- dzielenie przez $\sqrt{d_h}$ zapobiega zbyt dużym iloczynom (wtedy softmax byłby „twardy" i gradient znikałby);
- b_{j-i} — **wyuczona premia zależna od odległości** (względne kodowanie pozycji): sieć sama uczy się np. „pozycja o 12 półtonów wyżej jest szczególnie interesująca". Jej wartości to parametry; osobne dla każdej głowy;
- wagi $a_{ij} \geq 0$ sumują się do 1 po j, więc y_i jest **średnią ważoną** wartości.

Wiele głów. Wektor $D = 64$ dzielimy na $n_h = 4$ części po $d_h = 16$ i liczymy uwagę osobno w każdej części („każda głowa szuka czego innego"), po czym sklejamy wyniki i mnożymy przez macierz wyjściową W_O.

Przykład liczbowy (jedna głowa, $d_h = 2$). Zapytanie $q = (1, 0)$, klucze $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$, wartości $v_1 = (1, 0)$, $v_2 = (0, 2)$, $v_3 = (3, 3)$, bez premii: wyniki $q \cdot k/\sqrt{2} = (0{,}707;\ 0;\ 0{,}707)$ → wagi softmax $(0{,}401;\ 0{,}198;\ 0{,}401)$ (jak w M7) → wynik $y = 0{,}401\,(1, 0) + 0{,}198\,(0, 2) + 0{,}401\,(3, 3) = (1{,}604;\ 1{,}599)$.

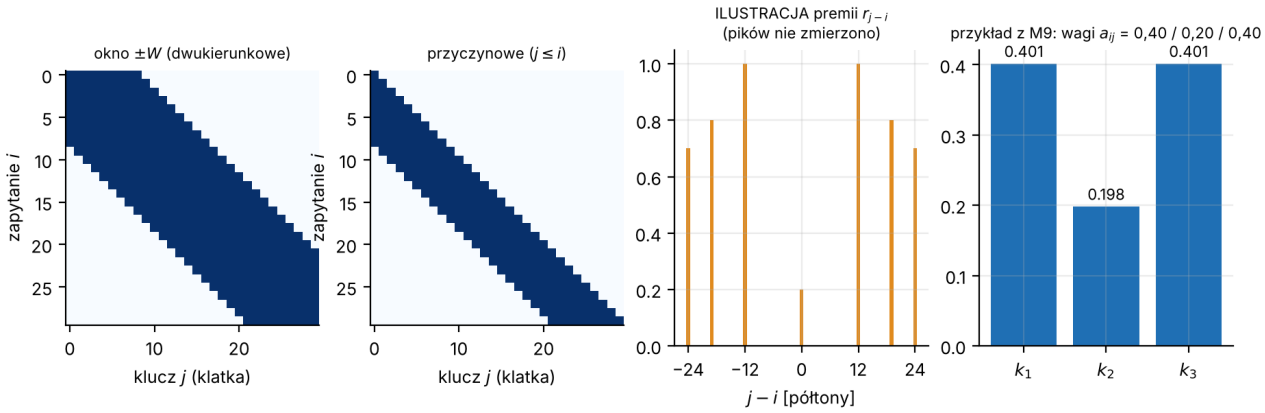
Czytaj to tak: zapytanie q jest podobne do kluczy k_1 i k_3 (oba mają „1" na pierwszej współrzędnej), więc wynik jest głównie mieszanką v_1 i v_3; v_2 dostało tylko 20 % wagi.

Przykład z modelu. Uwaga w czasie dla półtonu E4 w klatce $i = 100$: zapytanie q_{100} porównujemy z kluczami klatek 57–143 (okno ± 43). Jeśli w klatce 98 zaczęła się ta sama nuta, jej klucz jest podobny i dostaje dużą wagę, więc y_{100} „wie", że nuta trwa od dwóch klatek. Jeśli klucze



Rysunek 13: Uwaga w trzech krokach na minimalnym przykładzie (4 elementy, $d_h = 2$; liczby policzone naprawdę). Wyniki $q_i \cdot k_j/\sqrt{2}$ zamieniamy softmaxem na wagi (każdy wiersz sumuje się do 1) i mieszamy wartości v_j tymi wagami. Przykład: $y_1 = 0{,}35\,v_1 + 0{,}17\,v_2 + 0{,}35\,v_3 + 0{,}12\,v_4 \approx (1{,}53;\ 1{,}28)$.

klatek 101–110 wskazują ciszę, głowica końca może zareagować. Uwaga między wysokościami dla tej samej klatki porównuje E4 z E3 (odległość 12), A3 (odległość 7), A2 (odległość 19)...



Rysunek 14: Od lewej: maska okna dwukierunkowego (ciemne = dozwolone), maska przyczynowa (tylko przeszłość), **ilustracja** wyuczonej premii r_{j-i} w uwadze między wysokościami (piki w odległościach oktawy, kwinty-nad-oktawą i dwóch oktaw to przykład tego, czego sieć *może* się nauczyć; wartości są poglądowe i nie pochodzą z modelu), oraz wagi z przykładu liczbowego powyżej.

Maskowanie. Pozycje niedozwolone dostają wynik $-\infty$, więc po softmaksie waga $= 0$. Tak realizuje się:

- okno lokalne ($|j - i| \le W$),
- tryb **przyczynowy** (tylko $j \le i$: model „na żywo”, który nie zna przyszłości),
- ignorowanie dopełnienia (poza nagraniem).

Koszt. Pełna uwaga na T klatkach potrzebuje T^2 wag. Okno $\pm W$ zmniejsza to do $\approx T \cdot (2W+1)$. Implementacja (`model.py:LocalTimeAttention`) dzieli czas na bloki po W klatek; zapytania bloku patrzą na klucze trzech bloków (poprzedni, własny, następny), więc pamięć to $O(T \cdot 3W)$.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Co jest wynikiem uwagi, gdy wszystkie wyniki $q \cdot k$ są równe? **Odp.:** równe wagi $1/n$, czyli zwykła średnia wartości.
2. Co robi wynik $-\infty$? **Odp.:** po softmaksie daje wagę 0; tak działa maska okna i tryb przyczynowy.
3. Po co dzielimy przez $\sqrt{d_h}$? **Odp.:** żeby iloczyny skalarne nie rosły z wymiarem, bo duże wartości robią softmax „twardym" i gradient prawie znika.

Najczęstszy błąd: czytanie wag a_{ij} jako „prawdopodobieństwa, że nuta tam jest". To tylko *skąd czerpać informację*; decyzję podejmują dopiero głowice.

Co zapamiętać: Uwaga = ważona średnia wartości, z wagami zależnymi od podobieństwa zapytania do klucza (plus wyuczona premia za odległość).

M10. Funkcje straty: jak model „wie", że się myli

Po co nam to? Aby sieć mogła się uczyć, musi dostać jedną liczbę mówiącą „jak bardzo się mylisz". Strata jest tą liczbą.

Intuicja. Strata $\mathcal{L}$ to jedna liczba mierząca, jak bardzo przewidywania odbiegają od prawdy. Uczenie = szukanie parametrów, które ją minimalizują.

Entropia krzyżowa dwuklasowa (BCE). Dla przewidywanego prawdopodobieństwa $p = \sigma(z)$ i etykiety $y \in [0, 1]$:

$$\ell_{\text{BCE}}(p, y) = -\big[\, y \ln p + (1 - y) \ln(1 - p) \,\big].$$

Dla $y = 1$: kara $-\ln p$ (gdy $p = 0{,}9$: $0{,}105$; gdy $p = 0{,}2$: $1{,}609$ — pewny błąd boli bardzo). Dla $y = 0$: kara $-\ln(1 - p)$ (gdy $p = 0{,}1$: $0{,}105$). Etykiety **miękkie** ($y = 0{,}4$) też są dozwolone — wtedy minimum jest przy $p = y$.

Waga klasy pozytywnej α (`pos_weight`). Początków nut jest bardzo mało w porównaniu z klatkami ciszy (większość mapy to zera). Żeby model nie nauczył się „zawsze odpowiadaj 0", mnożymy człon pozytywny:

$$\ell_{\alpha}(p, y) = -\big[\, \alpha\, y \ln p + (1 - y) \ln(1 - p) \,\big].$$

W modelu $\alpha_{\text{onset}} = 4$, $\alpha_{\text{offset}} = 2$, $\alpha_{\text{frame}} = 1$. Przykład: $y = 1$, $p = 0{,}2$, $\alpha = 4$ → kara $4 \cdot 1{,}609 = 6{,}44$.

Średnia z maską. Nie każde miejsce ma etykietę (inne źródła danych mają inne adnotacje, a krótkie nagrania są dopełniane). Mnożymy stratę przez maskę $m \in \{0, 1\}$ i dzielimy przez liczbę aktywnych elementów:

$$\mathcal{L}_{\text{head}} = \frac{\sum m \cdot \ell}{\max(1, \sum m)}.$$

Błąd średniokwadratowy (MSE) dla głośności: $\ell = (\hat{v} - v)^2$, liczony tylko tam, gdzie głośność jest znana.

Entropia krzyżowa wieloklasowa (tabulatura): dla rozkładu softmax **p** po K klasach i prawdziwej klasy y:

$$\ell_{\mathrm{CE}} = -\ln p_y.$$

Klasy z etykietą -1 (nieznane) są pomijane (`ignore_index`).

Strata łączna to ważona suma głów (zob. rozdz. 6):

$$\mathcal{L} = w_{\mathrm{on}}\mathcal{L}_{\mathrm{on}} + w_{\mathrm{fr}}\mathcal{L}_{\mathrm{fr}} + w_{\mathrm{off}}\mathcal{L}_{\mathrm{off}} + w_{\mathrm{vel}}\mathcal{L}_{\mathrm{vel}} + w_{\mathrm{tab}}\mathcal{L}_{\mathrm{tab}}.$$

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Ile wynosi BCE dla $y = 1$, $p = 0{,}5$? **Odp.:** $-\ln 0{,}5 = 0{,}693$; z $\alpha = 4$ będzie $4 \cdot 0{,}693 = 2{,}77$.
2. Dlaczego $\alpha = 4$ dla początków? **Odp.:** początków jest bardzo mało, więc bez wagi sieć mogłaby wszędzie odpowiadać „nie".
3. Po co maska m? **Odp.:** by nie karać za miejsca, o których nie wiemy, jaka jest prawda (inne źródło danych, dopełnienie wycinka).

Najczęstszy błąd: porównywanie strat o różnych skalach „na oko". Dlatego w sumie łącznej każda część ma swoją wagę w.

Co zapamiętać: Strata = średnia kara za błędy; im pewniejszy błąd, tym większa kara.

M11. Optymalizacja: jak sieć się uczy

Po co nam to? Znamy już „co" (sieć) i „jak mierzymy błąd" (strata). Ten rozdział mówi, **jak poprawiać wagi**, żeby błąd malał.

Spadek gradientu. Parametry θ (wszystkie wagi) zmieniamy w kierunku **przeciwnym do gradientu** straty (gradient = wektor pochodnych cząstkowych $\partial\mathcal{L}/\partial\theta_i$, wskazuje kierunek najszybszego wzrostu):

$$\theta \leftarrow \theta - \eta\, g, \qquad g = \nabla_\theta \mathcal{L},$$

gdzie η to *tempo uczenia* (*learning rate*). Gradient liczy się **wsteczną propagacją**: reguła łańcuchowa $\frac{d}{dx} f(g(x)) = f'(g(x))\, g'(x)$ stosowana warstwa po warstwie od wyjścia do wejścia. Nie liczy się go z całych danych, tylko z małej **paczki** (*batch*, tu 16 wycinków po 192 klatki ≈ 4,4 s).

Adam / AdamW. Zwykły spadek gradientu ma kłopot, gdy różne parametry mają gradienty o bardzo różnej skali. Adam utrzymuje dla każdego parametru dwie średnie kroczące:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \boxed{\theta_t = \theta_{t-1} - \eta\Big(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda\, \theta_{t-1}\Big)}$$

- m_t — uśredniony kierunek (pęd, wygładza szum);

- v_t — uśredniony kwadrat gradientu (skala); dzielenie przez $\sqrt{\hat{v}_t}$ normalizuje krok: parametry o dużych gradientach dostają mniejsze kroki;
- $\hat{m}, \hat{v}$ — korekta obciążenia z początku uczenia (średnie startują od zera);
- domyślnie $\beta_1 = 0{,}9,\ \beta_2 = 0{,}999,\ \varepsilon = 10^{-8}$;
- $\lambda\theta$ — **rozpad wag** (*weight decay*) oddzielony od gradientu (to właśnie odróżnia **AdamW** od Adama): delikatnie ściąga wagi do zera, co działa jak regularyzacja. W modelu $\lambda = 10^{-4}$.

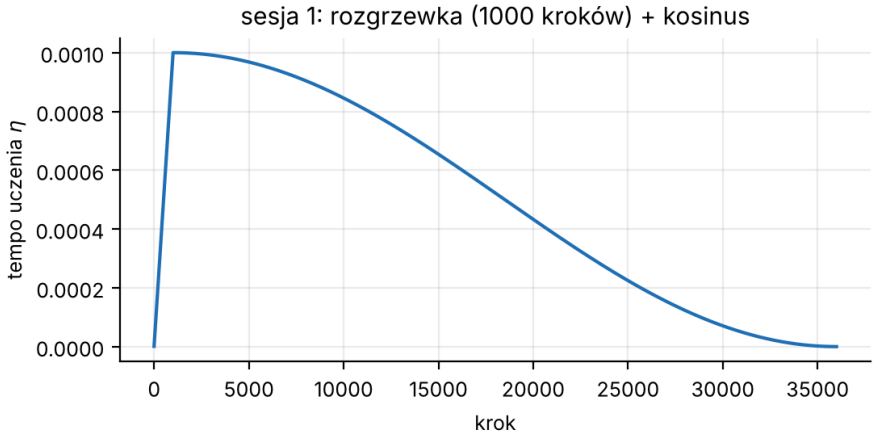
Przykład pierwszego kroku ($t = 1$, $g = 2$): $m_1 = 0{,}2,\ v_1 = 0{,}004$, po korekcie $\hat{m}_1 = 2,\ \hat{v}_1 = 4$, krok $= \eta \cdot 2/(2 + 10^{-8}) \approx \eta$. Czyli pierwszy krok ma zawsze długość $\approx \eta$, niezależnie od skali gradientu.

Harmonogram tempa uczenia.

- *Rozgrzewka* (liniowo od 0 do η_0 przez n_w kroków — na początku sieć jest losowa i duże kroki ją rozwalą): $\eta(s) = \eta_0 \frac{s+1}{n_w}$ dla $s < n_w$.
- *Kosinus* (łagodne wygaszanie do zera, Loshchilov i Hutter 2017): dla ułamka postępu $q \in [0, 1]$

$$\eta(q) = \eta_0 \cdot \tfrac{1}{2}(1 + \cos \pi q).$$

Wartości dla $\eta_0 = 10^{-3}$: $q = 0 \to 10^{-3}$; $0{,}25 \to 8{,}5 \cdot 10^{-4}$; $0{,}5 \to 5 \cdot 10^{-4}$; $0{,}75 \to 1{,}5 \cdot 10^{-4}$; $1 \to 0$.

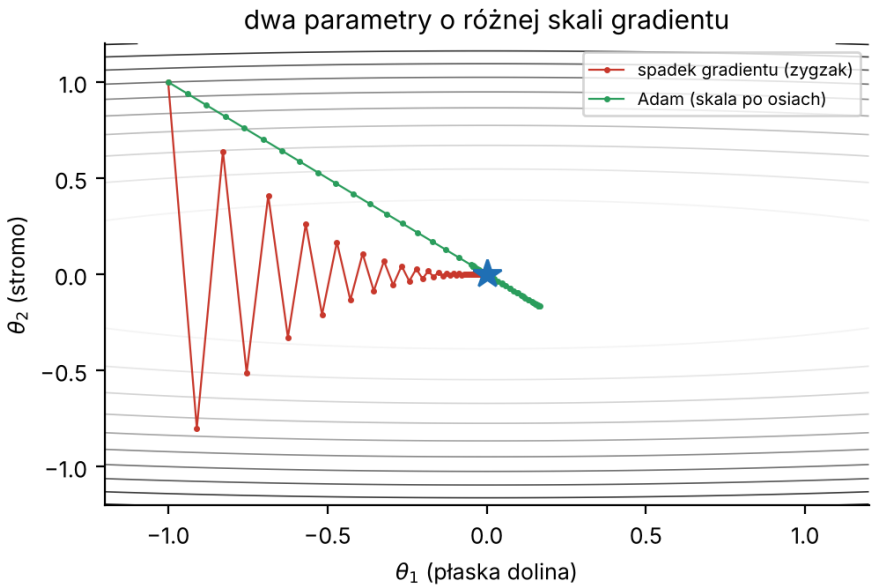


Rysunek 15: Tempo uczenia w sesji 1: liniowa rozgrzewka przez 1000 kroków do 10^{-3}, potem kosinus do zera po 36 000 krokach (36 epok po 1000 kroków).

Przykład użycia gradientu (uproszczony). Jedna waga $\theta = 0{,}50$, $\lambda = 10^{-4}$, $\eta = 10^{-3}$, po korekcie $\hat{m} = 0{,}2$ i $\hat{v} = 0{,}04$ (czyli $\sqrt{\hat{v}} = 0{,}2$). Krok: $\theta \leftarrow 0{,}50 - 10^{-3}\,(0{,}2/0{,}2 + 10^{-4} \cdot 0{,}5) = 0{,}50 - 10^{-3} \cdot 1{,}00005 = 0{,}49900$. Człon z $\hat{m}/\sqrt{\hat{v}}$ ma wielkość ≈ 1, więc waga zmienia się o $\approx \eta$, a rozpad wag dokłada tylko $5 \cdot 10^{-8}$.

Przycinanie gradientu. Jeśli norma całego gradientu $\|g\|_2$ przekracza próg $c = 1$, skalujemy go: $g \leftarrow g \cdot \min(1,\ c/\|g\|_2)$. Chroni przed pojedynczym „wybuchowym” krokiem.

Mieszana precyzja (AMP). Obliczenia w 16-bitowych liczbach zmiennoprzecinkowych (szybciej, mniej pamięci); strata jest mnożona przez duży współczynnik (*GradScaler*), by małe gradienty nie wyzerowały się w float16, a potem dzielona z powrotem.



Rysunek 16: Dlaczego Adam: funkcja straty o dwóch parametrach, z których jeden ma strome, a drugi płaskie zbocze (poziomice). Zwykły spadek gradientu (czerwony) zygzakuje w poprzek stromej osi, a Adam (zielony) dzieli krok przez skalę gradientu po każdej osi i idzie prościej do minimum (gwiazdka). To rysunek poglądowy, nie dane z uczenia modelu.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. Ile wynosi η po połowie kosinusowego harmonogramu dla $\eta_0 = 10^{-3}$? **Odp.:** $5 \cdot 10^{-4}$.
2. Co robi przycinanie gradientu przy $\|g\|_2 = 5$ i progu 1? **Odp.:** skaluje gradient przez $1/5$, więc jego norma staje się 1.
3. Czym AdamW różni się od Adama? **Odp.:** rozpad wag $\lambda\theta$ jest dodawany *osobno*, poza adaptacyjnym skalowaniem gradientu.

Najczęstszy błąd: myślenie, że większe tempo uczenia zawsze uczy szybciej. Za duże powoduje skakanie wokół minimum albo rozbieżność, dlatego jest rozgrzewka i wygaszanie.

Co zapamiętać: Uczenie = wiele małych kroków w stronę mniejszej straty; Adam dobiera wielkość kroku osobno dla każdej wagi.

M12. Dekodowanie: od map prawdopodobieństw do listy nut

Po co nam to? Sieć daje mapy prawdopodobieństw, a użytkownik chce listę nut. Dekoder jest „tłumaczem” z jednego na drugie i działa na prostych regułach.

Sieć zwraca **mapy** $p_{\text{on}}(t,s)$, $p_{\text{fr}}(t,s)$, $p_{\text{off}}(t,s) \in (0,1)$ — dla każdej klatki i półtonu. Dekoder (deterministyczny algorytm bez uczenia) zamienia je w nuty za pomocą **progów** θ i **lokalnych maksimów**:

$$\text{pik w czasie:} \quad p(t,s) \geq \theta, \;\; p(t,s) \geq p(t-1,s), \;\; p(t,s) > p(t+1,s).$$

(Lokalne maksimum, żeby szerokie „wzgórze” prawdopodobieństwa dało **jedną** nutę, nie kilka. Warunek $\geq$ z lewej i $>$ z prawej wybiera pierwszy z równych szczytów.) Szczegóły algorytmu w rozdz. 8.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. W ciągu $p_{\text{on}} = (0{,}2;\ 0{,}7;\ 0{,}65;\ 0{,}5)$ przy progu $\theta = 0{,}6$ gdzie jest pik? **Odp.:** w drugim elemencie (0,7: $\geq 0{,}6$ i większe od sąsiadów 0,2 i 0,65).
2. Dlaczego potrzebujemy lokalnego maksimum, a nie tylko progu? **Odp.:** bez tego szeroki garb prawdopodobieństwa dałby kilka nut zamiast jednej.
3. Czy dekoder jest uczony gradientem? **Odp.:** nie, jego progi są hiperparametrami strojonymi na walidacji.

Najczęstszy błąd: ustawianie progów na zbiorze testowym. To „oszukiwanie” (przeciek informacji); progi strojono wyłącznie na walidacji.

Co zapamiętać: Dekoder = progi + lokalne maksima, bez uczenia.

M13. Miary jakości

Po co nam to? Aby powiedzieć, czy model jest dobry, i porównać go uczciwie z innym. Każda miara odpowiada na inne pytanie.

Precyzja, czułość, F1. Gdy porównujemy listę nut systemu z wzorcem: TP (prawdziwie pozytywne: trafione nuty), FP (fałszywe alarmy: nuty systemu bez odpowiednika), FN (pominięte: nuty wzorca bez odpowiednika).

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R} = \frac{2\,TP}{2\,TP + FP + FN}.$$

Precyzja = „jaka część moich nut jest dobra”, czułość = „jaką część prawdziwych nut znalazłem”, F1 = średnia harmoniczna (karze, gdy któraś jest mała). Przykład: $TP = 80, FP = 20, FN = 20 \Rightarrow P = R = F_1 = 0{,}8$.

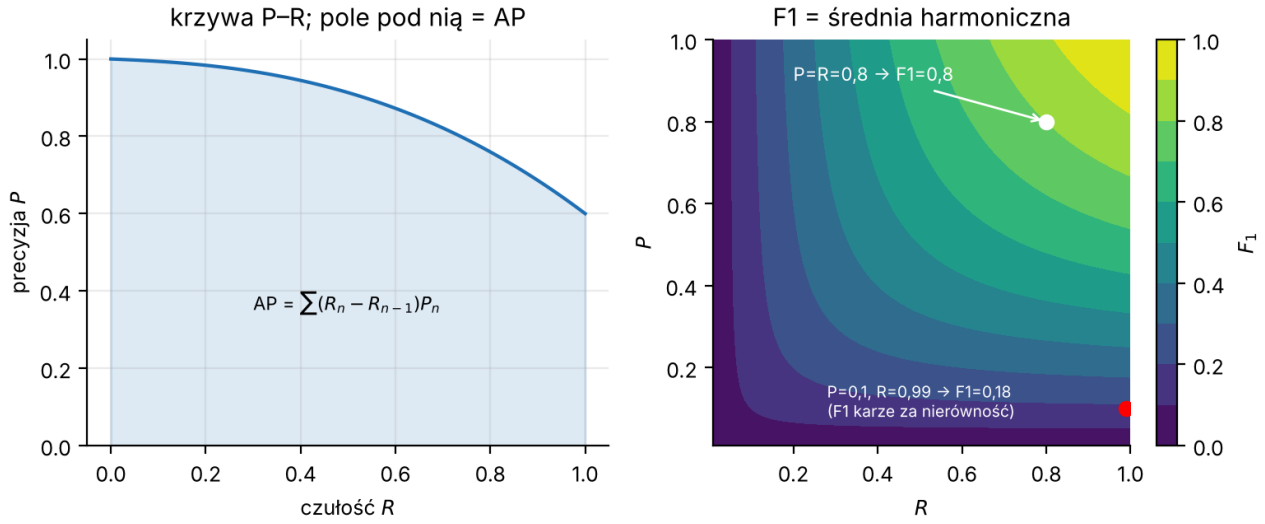
Kiedy nuta jest „trafiona” (standard `mir_eval`, Raffel i in. 2014). Para (nuta wzorca i, nuta systemu j) jest *dopuszczalna*, gdy jednocześnie:

1. $|\text{początek}_i - \text{początek}_j| \leq 50$ ms,
2. $\left|1200 \log_2 \frac{f_i}{f_j}\right| \leq 50$ ¢ (czyli tej samej wysokości z dokładnością do ćwierć tonu),
3. *(tylko w mierze „z końcem”)* $|\text{koniec}_i - \text{koniec}_j| \leq \max\left(50 \text{ ms},\ 0{,}2 \cdot (\text{koniec}_i - \text{początek}_i)\right)$.

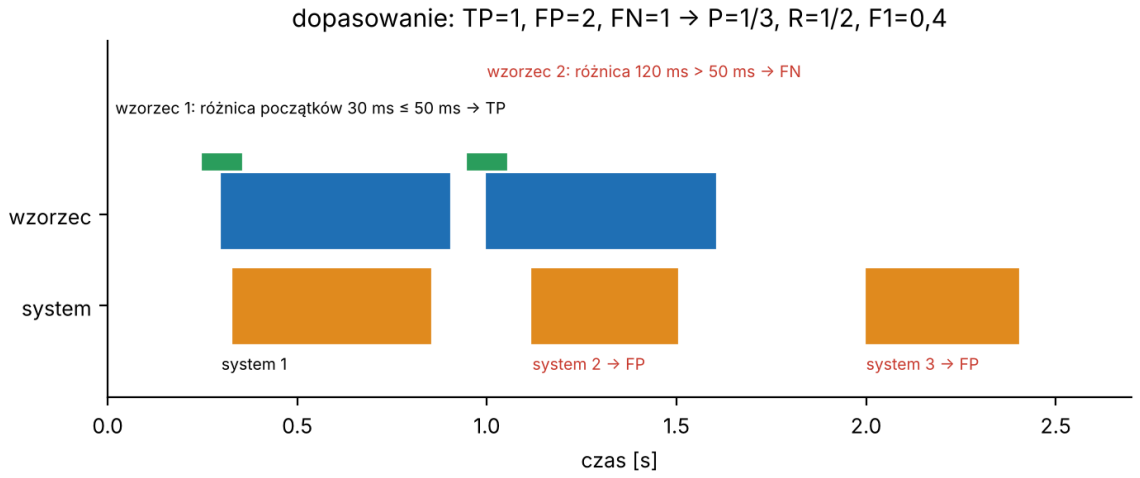
Każda nuta może należeć do co najwyżej jednej pary, więc wybieramy **maksymalne skojarzenie** w grafie dwudzielnym algorytmem Hopcrofta–Karpa (1973), złożoność $O(E\sqrt{V})$. (50 ms to 2,15 klatki modelu — dlatego krok 23 ms zamiast 46 ms miał znaczenie.)

Miary klatkowe. Czas dzielimy na siatkę co 10 ms; w każdej komórce i dla każdej wysokości jest „brzmi/nie brzmi”. Wtedy TP, FP, FN liczymy po komórkach, a ponadto (Poliner i Ellis 2007) Acc $= \frac{TP}{TP+FP+FN}$ i błędy względem liczby nut wzorca $N_{\text{ref}} = \sum_t n_{\text{ref}}(t)$:

$$E_{\text{sub}} = \frac{\sum_t [\min(n_{\text{ref}}, n_{\text{est}}) - n_{\text{cor}}]}{N_{\text{ref}}}, \quad E_{\text{miss}} = \frac{\sum_t \max(0, n_{\text{ref}} - n_{\text{est}})}{N_{\text{ref}}}, \quad E_{\text{fa}} = \frac{\sum_t \max(0, n_{\text{est}} - n_{\text{ref}})}{N_{\text{ref}}},$$



Rysunek 17: Lewo: krzywa precyzja–czułość (przykład poglądowy); pole pod nią to średnia precyzja AP. Prawo: wartość F_1 jako funkcja P i R: wysoka tylko wtedy, gdy obie są wysokie. Przy $P = 0{,}1,\ R = 0{,}99$ wychodzi $F_1 = 0{,}18$, bo średnia harmoniczna „karze” za nierówność.



Rysunek 18: Przykład dopasowania. Wzorzec 1 (niebieski) i system 1 (pomarańczowy) mają początki oddalone o 30 ms, więc to trafienie (TP). Wzorzec 2 ma najbliższą nutę systemu 120 ms dalej, więc jest pominięty (FN), a nuty systemu 2 i 3 to fałszywe alarmy (FP). Stąd $P = 1/3$, $R = 1/2$, $F_1 = 2 \cdot \frac{1}{3} \cdot \frac{1}{2} / (\frac{1}{3} + \frac{1}{2}) = 0{,}4$.

$$E_{\text{tot}} = \frac{\sum_t [\max(n_{\text{ref}}, n_{\text{est}}) - n_{\text{cor}}]}{N_{\text{ref}}} = E_{\text{sub}} + E_{\text{miss}} + E_{\text{fa}}.$$

Średnia precyzja (AP) — jakość samych map prawdopodobieństw, bez wyboru progu. Sortujemy wszystkie punkty malejąco po prawdopodobieństwie; po zejściu do n-tego punktu mamy precyzję P_n i czułość R_n (jakby próg leżał tuż pod nim):

$$\text{AP} = \sum_n (R_n - R_{n-1})\, P_n$$

(pole pod krzywą precyzja–czułość).

Kalibracja (ECE) i Brier. Czy „0,8" znaczy naprawdę 80 %? Dzielimy przewidywania na $B = 15$ przedziałów po $1/15$; w każdym porównujemy średnie przewidywanie z faktyczną częstością:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N} \left| \overline{y}(\mathcal{B}_b) - \overline{p}(\mathcal{B}_b) \right|, \qquad \text{Brier} = \frac{1}{N} \sum_i (p_i - y_i)^2.$$

Mikro i makro. *Mikro*: sumujemy TP/FP/FN po wszystkich plikach i liczymy jedno F1 (długie pliki ważą więcej). *Makro*: liczymy F1 osobno dla każdego pliku i uśredniamy (każdy plik waży tyle samo; tak raportuje się w publikacjach).

Przedział ufności bootstrap. Mamy wyniki $x_1, \ldots, x_n$ po plikach. Losujemy n plików **ze zwracaniem**, liczymy średnią; powtarzamy B razy (2000). Środkowe 95 % tych średnich (percentyle 2,5 i 97,5) to przedział ufności średniej. **Sparowany** bootstrap porównuje system A i B na *tych samych* plikach: bootstrapuje różnice $d_i = a_i - b_i$; jeśli przedział różnicy nie zawiera zera (a $p = \Pr(\bar{d}^* \le 0)$, czyli odsetek losowań bootstrapu, w których średnia różnica wyszła ≤ 0, jest małe), poprawa jest istotna statystycznie.

Sprawdź siebie. Spróbuj sam, potem zajrzyj do odpowiedzi.

1. $TP = 30,\ FP = 10,\ FN = 20$. Ile wynoszą P, R, F_1? **Odp.:** $P = 0{,}75,\ R = 0{,}6,\ F_1 = 60/90 = 0{,}667$.
2. Czym różni się mikro od makro? **Odp.:** mikro sumuje nuty ze wszystkich plików, makro uśrednia wyniki plików (każdy plik waży tyle samo).
3. Co znaczy, że przedział bootstrap różnicy $[0{,}46;\ 0{,}83]$ nie zawiera 0? **Odp.:** poprawa jest istotna statystycznie.

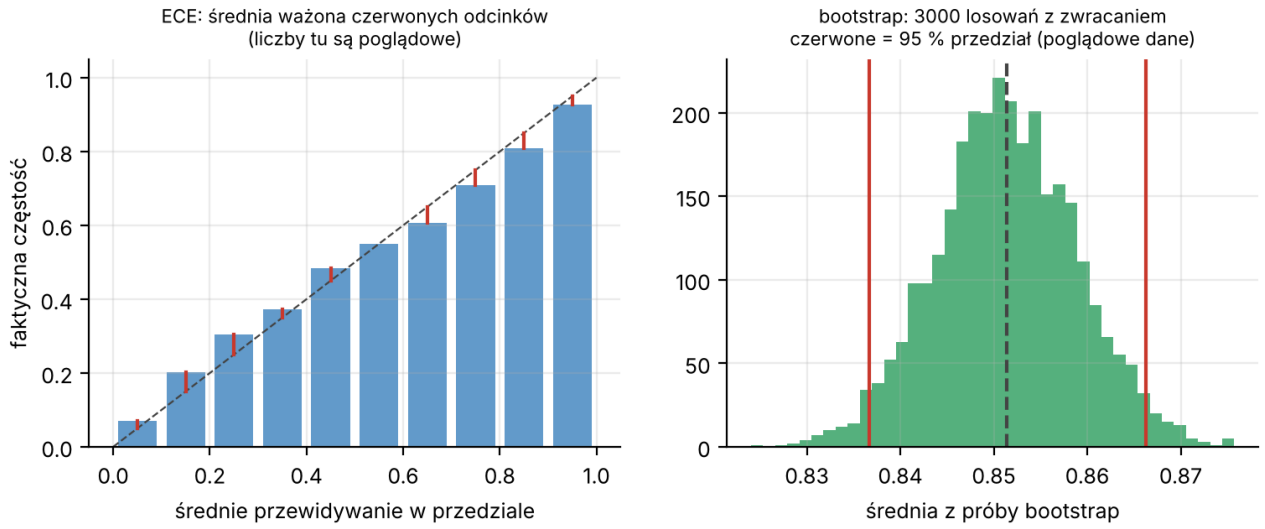
Najczęstszy błąd: uznanie wyniku bez przedziału ufności za pewny. Różnica 0,3 pkt na 12 plikach może być tylko szumem.

Co zapamiętać: F1 łączy precyzję i czułość, a przedział ufności mówi, ile możemy temu wynikowi ufać.

M14. Jak to wszystko się łączy (mapa myślowa)

$$\underbrace{x[n]}_{\text{M1}} \xrightarrow[\text{M4, M5}]{\text{FFT + max + dB}} \underbrace{X \in \mathbb{R}^{T\times 9\times 159}}_{\text{HCQT}} \xrightarrow[\text{M6, M3}]{\text{konwolucje harmoniczne}} \underbrace{h \in \mathbb{R}^{T\times 53\times 64}}_{\text{cechy półtonów}} \xrightarrow[\text{M9, M8}]{\text{uwaga}} \text{mapy } p \xrightarrow[\text{M12}]{\text{dekoder}} \text{nuty} \xrightarrow[\text{M13}]{\text{F1}} \text{wynik}$$

Uczenie: M10 (strata) + M11 (AdamW, harmonogram).



Rysunek 19: Lewo: wykres kalibracji (poglądowy). Gdyby „0,8” oznaczało zawsze 80 %, słupki leżałyby na przerywanej przekątnej; ECE to średnia ważona długości czerwonych odcinków. Prawo: bootstrap: wielokrotnie losujemy pliki z zwracaniem i liczymy średnią; środkowe 95 % tych średnich to przedział ufności. Dane obu paneli są syntetyczne, służą tylko do zrozumienia metody.

CZĘŚĆ II. PRACA NAUKOWA

1. Wstęp

Zadanie. *Automatyczna transkrypcja muzyki* (AMT) polega na zamianie nagrania na zapis nutowy. Dla gitary dochodzą dwie trudności: (i) **wielogłosowość** — akord to 3–6 nut naraz, a alikwoty jednej nuty zlewają się z podstawami innych (błąd oktawowy; w naszych wynikach 24 % fałszywych alarmów klatkowych wypada w oktawie od prawdziwej nuty); (ii) **niejednoznaczność tabulatury** — ta sama wysokość da się zagrać na kilku strunach (np. E4 to struna 1 próg 0, struna 2 próg 5, struna 3 próg 9 itd.), a to, na której, wynika z brzmienia, a nie z samej wysokości.

Wkład Gen 6. (1) Cechy HCQT z krokiem 23 ms (poprzednio 46 ms) i dokładną zmianą próbkowania. (2) *Blok harmoniczny* — warstwa, która w każdym binie widzi jednocześnie bin własny oraz biny na alikwotach 0,5×, 1×, …, 6× (rozdz. 5.2). (3) Uwaga w czasie dwukierunkowa i **uwaga między wysokościami** (rozdz. 5.4–5.5). (4) Pięć głowic, w tym koniec nuty i tabulatura, z **maskami dostępności etykiet**, co pozwala uczyć na danych o różnych adnotacjach (rozdz. 6). (5) Dekoder z progami strojonymi na walidacji (rozdz. 8). (6) Pełna ewaluacja zgodna z `mir_eval` z przedziałami bootstrap (rozdz. 9).

2. Prace pokrewne

- **Cechy harmoniczne.** Bittner i in. (2017) wprowadzili HCQT jako wejście sieci do estymacji f0: warstwy transformaty o stałym ilorazie Q, w których kanały są przeskalowane o

całkowite alikwoty. Gen 6 przejmuje tę ideę, ale licząc z FFT i dołączając alikwot 0,5×.

- **Transkrypcja fortepianu.** Hawthorne i in. (2018) w *Onsets and Frames* przewidują osobno początki nut i ramki, a głowica ramek dostaje wyjście głowicy początków. Gen 6 stosuje to samo sprzężenie (`onset_to_frame`).
- **Gitara.** GuitarSet (Xi i in. 2018) to nagrania z przetwornikiem heksafonicznym i adnotacją strunową; Wiggins i Kim (2019) zaproponowali miarę TDR dla tabulatury.
- **Uwaga.** Vaswani i in. (2017); względne premie pozycyjne: Shaw i in. (2018).
- **Pozostałe składniki:** LayerNorm (Ba i in. 2016), GELU (Hendrycks i Gimpel 2016), AdamW (Loshchilov i Hutter 2019), harmonogram kosinusowy (Loshchilov i Hutter 2017), konwolucje dylatowane (Yu i Koltun 2016), ewaluacja `mir_eval` (Raffel i in. 2014), bootstrap (Efron 1979).

3. Dane i etykiety

3.1. Zbiory

zbiór	nagrania	opis
klasyka, zestaw 1: uczenie / walidacja / test	5502 / 450 / 520	utwory z korpusu MIDI (MAESTRO v3 + music21) zagrane syntezatorem *gen_classical* na 27 instrumentach, 11 miejscach (pogłos), 4 formach utworu; utwór zawsze w jednym zbiorze
klasyka, zestaw 2	9135 (tylko uczenie)	inne ziarna losowe: transpozycje, tempa, obsady, miejsca
środowiska gitarowe	888	gitara w losowym pomieszczeniu z szumem, brumem i stukami
GuitarSet (prawdziwa gitara)	168 / 12 / 180	gracze 00–02 uczenie, funk gracza 02 walidacja, gracze 03–05 test

Razem ok. 16 850 nagrań; do uczenia ok. 15 600 (ok. 56 h). Test: **700 nagrań** (180 GuitarSet + 520 klasyki) — utwory i gracze niewidziani w uczeniu. Z tego **tylko GuitarSet jest prawdziwą gitarą**; większość danych jest syntetyczna (to ważne dla oceny wniosków, rozdz. 11).

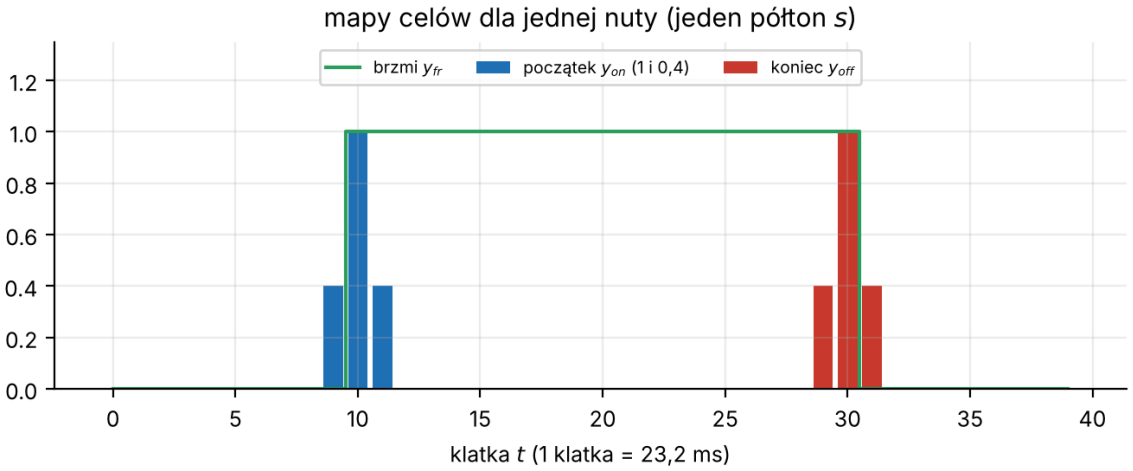
3.2. Mapy celów

Dla nuty o wysokości MIDI m, początku t_{on}, końcu t_{off} (i ewentualnie końcu słyszalnym t_a: $t_{\text{end}} = \min(t_{\text{off}}, t_a)$) definiujemy półton $s = m - 36$ oraz mapy (τ_t — środek klatki, M4; $\Delta = 256/f_s$ — czas klatki):

- **brzmi:** $y_{\text{fr}}(t, s) = 1 \Leftrightarrow t_{\text{on}} - \frac{1}{2}\Delta \le \tau_t \le \max(t_{\text{end}}, t_{\text{on}})$. Dolna granica o pół klatki wcześniej gwarantuje, że klatka początku zawsze też „brzmi” (poprawka względem Gen 5, gdzie w tej klatce etykieta bywała 0, mimo że dekoder wymaga $p_{\text{fr}} \ge 0{,}3$);

- **początek:** $k = \text{round}\big((t_{\text{on}} f_s - N_L/2)/256\big)$; $y_{\text{on}}(k, s) = 1$, a w klatkach $k \pm 1$ wartość **miękka** $0{,}4$ (w razie nakładania bierze się maksimum). Miękkie etykiety nie karzą sieci za niepewność ±1 klatki (±23 ms);
- **koniec:** jak początek, dla t_{off};
- **głośność:** $y_{\text{vel}}(k, s) =$ głośność 0–1 w klatce początku, z maską $= 1$ tylko tam, gdzie jest znana;
- **tabulatura:** dla struny i klasa $c_i(t) \in \{0, 1, \ldots, 20\}$: 0 = struna milczy, $1 + f$ = gra próg $f \in \{0, \ldots, 19\}$, gdzie $f = m - o_i$ i o_i to wysokość struny pustej. Nieznane: -1.

Przykład (jedna nuta). Nuta E4 ($m = 64 \Rightarrow s = 28$) zaczyna się w $t_{\text{on}} = 1{,}000$ s i kończy w $t_{\text{off}} = 1{,}400$ s. Klatka początku: $k = \text{round}\left((1{,}0 \cdot 11025 - 2048)/256\right) = \text{round}(35{,}07) = 35$. Mapa początku w kolumnie $s = 28$: klatki 34, 35, 36 mają wartości $0{,}4;\ 1;\ 0{,}4$, reszta 0. Mapa „brzmi”: $\tau_t \geq 1{,}0 - 0{,}0116 = 0{,}988$ s, czyli od $t = 35$ (środek $0{,}998$ s), do $\tau_t \leq 1{,}4$ s, czyli do $t = 52$ ($\tau_{52} = 1{,}3932$ s; klatka 53 ma $1{,}4166$ s, więc już nie). Koniec: $k_{\text{off}} = \text{round}((1{,}4 \cdot 11025 - 2048)/256) = \text{round}(52{,}29) = 52$, wartości $0{,}4; 1; 0{,}4$ w klatkach 51, 52, 53. Jeśli to nuta ze struny 2 ($o = 59$), próg to $64 - 59 = 5$, więc klasa tabulatury $= 1 + 5 = 6$ na strunie 2 w klatkach 35–52, a na pozostałych strunach 0 (milczą).



Rysunek 20: Mapy celów dla jednej nuty: „brzmi” (zielony schodek), początek (niebieski: 1 w klatce początku i miękkie 0,4 po obu stronach) i koniec (czerwony). Miękkie sąsiedztwo oznacza, że przewidzenie początku o jedną klatkę (23 ms) obok nie jest karane tak mocno jak całkowite chybienie.

3.3. Dostępność etykiet

Każde nagranie niesie wektor dostępności $\mathbf{a} \in \{0, 1\}^5$ (które z głowic: początek, brzmi, koniec, głośność, tabulatura ma *naprawdę* zaadnotowane). Strata (rozdz. 6) mnoży błąd głowicy przez a_{head}. Dzięki temu można mieszać źródła, np. GuitarSet (ze strunami, bez głośności) i MIDI (bez strun).

4. Cechy wejściowe

Wejściem jest tensor HCQT $X \in \mathbb{R}^{T \times 9 \times 159}$ zdefiniowany w M4–M5. Parametry:

parametr	wartość	uzasadnienie
f_s	11 025 Hz	dokładna zmiana próbkowania (poprzednio silnik C++ dzielił przez liczbę całkowitą, więc przy 48 kHz dawał 12 kHz i inne biny)
krok	256 próbek = 23,2 ms	tolerancja początku w ewaluacji to 50 ms = 2,15 klatki
okna	4096 (372 ms), 1024 (93 ms)	rozdzielczość tonów (4,9 Hz przy E2) vs rozdzielczość czasu
zero-padding	$\times 4$	gładkie położenie piku
zakres	MIDI 36–88, 53 półtony	najniższa struna (drop C) – ok. 24. próg wysokiej E
biny	3 na półton (159)	tolerancja rozstrojenia ±33 ¢
alikwoty	długie: 0,5; 1; 2; 3; 4; 5. krótkie: 1; 2; 3	M3
normalizacja	99,5. percentyl, zakres 60 dB, przycięcie 1,3	niezależność od głośności nagrania

5. Architektura sieci

5.1. Przepływ tensorów

Niech B — rozmiar paczki. Kolejne kształty (`model.py:Gen6.encode/forward`):

etap	kształt	operacja
wejście	$[B,\ 9,\ T,\ 159]$	HCQT
trzon	$[B,\ 48,\ T,\ 159]$	konwolucja 3×7 + GELU
3 bloki harmoniczne	$[B,\ 48,\ T,\ 159]$	rozdz. 5.2
zejście	$[B,\ T,\ 53,\ 64]$	konwolucja 1×3, krok 3, + osadzenie wysokości
2 warstwy kontekstu	$[B,\ T,\ 53,\ 64]$	uwaga w czasie + między wysokościami + FF
LayerNorm	$[B,\ T,\ 53,\ 64]$	
głowice	początek/brzmi/koniec/głośność $[B, T, 53]$; tabulatura $[B, T, 6, 21]$	rozdz. 5.6

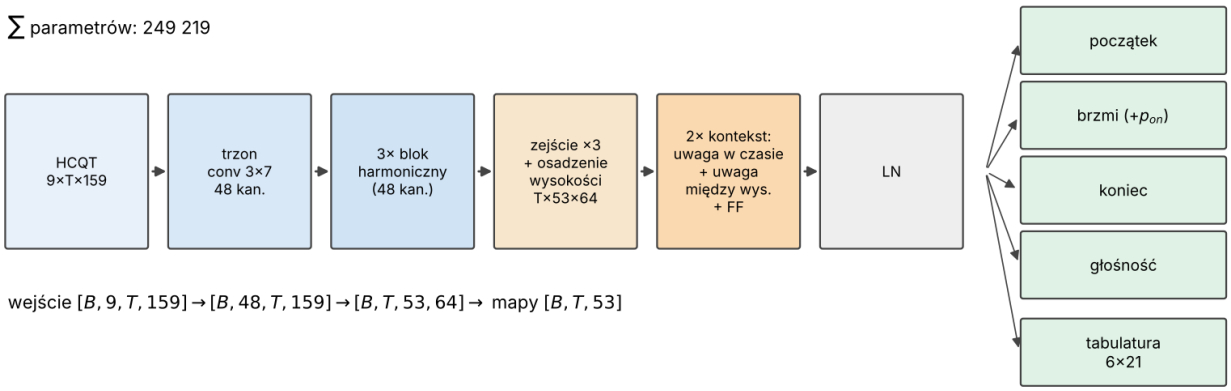
Łącznie **249 219 parametrów** (rozbicie w dodatku A).

5.2. Blok harmoniczny

To najważniejsza nowość. Niech $x \in \mathbb{R}^{H \times T \times P}$ (wejście bloku) i $K = \{-36, 0, 36, 57, 72, 84, 93\}$ — zestaw przesunięć z M3. Zdefiniujmy przesunięcie osi binów

$$\text{shift}_k(h)[\cdot, t, p] = h[\cdot, t, p + k] \quad (\text{zero poza zakresem}).$$

Blok liczy:



Rysunek 21: Schemat sieci Gen 6 z kształtami tensorów. Pięć głowic czyta te same cechy $h_{t,s} \in \mathbb{R}^{64}$; głowica „brzmi” dostaje dodatkowo p_{on} (bez przekazywania gradientu).

$$\tilde{h} = \mathrm{LN}_{\text{kan}}(x), \qquad u = W_h \left[\mathrm{shift}_{k_1}(\tilde{h}) \,\|\, \cdots \,\|\, \mathrm{shift}_{k_7}(\tilde{h}) \right],$$

$$v = \mathrm{GELU}\big(\mathrm{DW}_{1\times 5}(\mathrm{GELU}(u)) \big), \qquad w = \mathrm{Conv}^{(d)}_{3\times 1}(v), \qquad \boxed{y = x + \mathrm{Drop}\left(\mathrm{GELU}(w) \right)}$$

- $\|$ — sklejenie wzdłuż osi kanałów (7 kopii po $H = 48$ daje 336 kanałów);
- W_h — konwolucja 1×1 z $336 \to 48$ kanałów (16 176 parametrów);
- **sens:** w pozycji p (kandydat na f_0) sieć dostaje *jednocześnie* energię z 7 miejsc: subharmonicznej, podstawy i alikwotów 2–6. W_h uczy się „wzorca nuty”. Skoro oś jest logarytmiczna, wzorzec jest ten sam dla każdej wysokości (M3). Poprzednia generacja miała tylko konwolucje 3×7 i jedną warstwę „oktawa wyżej/niżej”;
- $\mathrm{DW}_{1\times 5}$ — filtr głębokościowy po sąsiednich binach (M6): rozstrojenie i nieharmoniczność (sztywna struna ma alikwoty nieco wyżej niż hf_0);
- $\mathrm{Conv}^{(d)}_{3\times 1}$ — filtr w czasie z dylatacją $d \in \{1, 2, 4\}$ w kolejnych blokach.

**P

5.3. Zejście do półtonów i osadzenie wysokości

Konwolucja 1×3 z krokiem 3 sprowadza 159 binów do 53 półtonów (wyjście s widzi biny $3s, 3s+1, 3s+2$) i zwiększa liczbę kanałów z 48 do $D = 64$. Do wyniku dodajemy wyuczony wektor $e_s \in \mathbb{R}^{64}$ dla każdego półtonu (*osadzenie wysokości*, $53 \cdot 64 = 3392$ parametrów):

$$h^{(0)}_{t,s} = \mathrm{Conv}_{1\times 3,\ \text{stride } 3}(y)_{t,s} + e_s.$$

Dzięki e_s sieć wie, w którym rejestrze jest kandydat (niskie nuty gitary zachowują się inaczej niż wysokie).

**P

5.4. Uwaga w czasie

Dla każdej wysokości s osobno, sekwencja $h_{1,s}, \dots, h_{T,s}$ przechodzi przez uwagę (M9) z oknem $|j-i| \le W = 43$ klatek ($43 \cdot 23{,}2$ ms ≈ 1 s) w obie strony:

$$a_{ij}^{(\text{time})} = \text{softmax}_{j:\,|j-i|\le W}\left(\frac{q_i \cdot k_j}{\sqrt{d_h}} + b_{j-i}^{(\text{time})}\right), \qquad d_h = D/n_h = 16.$$

Premia $b^{(\text{time})}$ ma $2W+1 = 87$ wartości na głowę. Cel: zdecydować, czy nuta „trwa dalej", jest ponownym szarpnięciem, czy kończy się, na podstawie kontekstu ±1 s. Wariant przyczynowy (`causal=True`) dopuszcza tylko $j \le i$ (tryb na żywo). Poprzednia generacja miała uwagę wyłącznie wstecz, choć analizowała całe nagranie.

**P

5.5. Uwaga między wysokościami (nowość)

W jednej klatce t traktujemy 53 półtony jako sekwencję i stosujemy uwagę (M9) **bez okna** (53 elementy):

$$a_{ij}^{(\text{pitch})} = \text{softmax}_{j}\left(\frac{q_i \cdot k_j}{\sqrt{d_h}} + r_{\,j-i}\right), \qquad j-i \in \{-52, \dots, 52\}.$$

Premia r ma $2S-1 = 105$ wartości na głowę; może się nauczyć, że odległości 12 (oktawa), 19 (oktawa + kwinta), 24 itd. są wyjątkowe — dokładnie te, w których alikwoty jednej nuty zlewają się z innymi nutami. Sieć ma więc mechanizm, by rozstrzygnąć „ten pik w $2f_0$ jest alikwotem nuty niżej czy osobną nutą". Uwaga ta widzi też **akord jako całość**.

**P

5.6. Warstwa kontekstu i pole widzenia

Jedna warstwa kontekstu (pre-norm, M8) liczy kolejno:

$$h' = h + \text{Drop}\big(\,\text{TA}(\text{LN}(h))\big), \quad h'' = h' + \text{Drop}\big(\,\text{PA}(\text{LN}(h'))\big), \quad h''' = h'' + \text{Drop}\big(\,\text{FF}(\text{LN}(h''))\big),$$

gdzie TA — uwaga w czasie, PA — uwaga między wysokościami, a $\text{FF} = W_2\,\text{GELU}(W_1 z)$ z $W_1 : \mathbb{R}^{64} \to \mathbb{R}^{256}$, $W_2 : \mathbb{R}^{256} \to \mathbb{R}^{64}$ (rozszerzenie ×4 jest standardem w transformerach). Warstwy są dwie.

Pole widzenia w czasie (ile klatek z każdej strony wpływa na wynik): trzon i bloki harmoniczne: $1 + (1+2+4)\cdot 1 = 8$; dwie warstwy kontekstu: $2 \cdot 2 \cdot 43 = 172$. Razem

$$\text{halo} = 8 + 172 = 180 \text{ klatek} \approx 4{,}2 \text{ s}.$$

Wnioskowanie na całym nagraniu (`Gen6.predict`) dzieli je na kawałki po 512 klatek z zakładką 180 klatek z każdej strony i odrzuca zakładkę, więc wynik **nie zależy od długości kawałka**

(sprawdza to test).

**P

5.7. Głowice

Wszystkie operują na $h_{t,s} = \mathrm{LN}(h^{(2)}_{t,s}) \in \mathbb{R}^{64}$. Każda to małe MLP $\mathrm{MLP}(z) = W_2\,\mathrm{GELU}(W_1 z)$:

$$p_{\text{on}} = \sigma(\,\mathrm{MLP}_{\text{on}}(h)), \qquad p_{\text{fr}} = \sigma(\,\mathrm{MLP}_{\text{fr}}([\,h\,;\ \mathrm{sg}(p_{\text{on}})\,])),$$
$$p_{\text{off}} = \sigma(\,\mathrm{MLP}_{\text{off}}(h)), \qquad \hat{v} = \sigma(\,\mathrm{MLP}_{\text{vel}}(h)),$$

gdzie $[\,\cdot\,;\,\cdot\,]$ to sklejenie (65 wejść do głowicy „brzmi"), a $\mathrm{sg}(\cdot)$ — *stop-gradient* (`.detach()`): głowica „brzmi" dostaje p_{on} jako cechę, ale jej strata nie zmienia głowicy początku (jak w *Onsets and Frames*).

Obciążenia ostatnich warstw są zainicjowane na -3 (początek, koniec) i -2 (brzmi): na starcie $\sigma(-3) = 0{,}047$ i $\sigma(-2) = 0{,}119$, czyli sieć od początku „spodziewa się", że nut jest mało, zamiast zaczynać od 0,5 i mozolnie się tego uczyć.

Głowica tabulatury. Dla struny $i \in \{1..6\}$ z pustą wysokością o_i i progu $f \in \{0..19\}$ odpowiedni półton to $s_{i,f} = o_i + f - 36$ (o ile mieści się w $[0, 52]$; reszta jest maskowana jako $-\infty$). Logit:

$$g_{i,f}(t) = w_i^{\top} h_{t,\,s_{i,f}} + c_i + \beta_{i,f},$$

gdzie $w_i \in \mathbb{R}^{64}$, c_i — wagi i obciążenie *osobne dla struny*, a $\beta_{i,f}$ — wyuczona premia za próg (wysokie progi są rzadsze). Klasę „struna milczy" dostaje się z maksimum cech po półtonach, które ta struna może zagrać:

$$g_{i,\emptyset}(t) = \mathrm{MLP}_{\text{sil}}\Big(\max_{f}\, h_{t,s_{i,f}}\Big) \quad (\text{maksimum po } f \text{ dla każdego kanału osobno}).$$

Wynik: wektor 21 logitów na strunę, a $p_i = \mathrm{softmax}(g_{i,\emptyset}, g_{i,0}, \ldots, g_{i,19})$ (M7). Ten projekt ma zaletę: logit „struna i gra próg f" jest liczony z cech **tego półtonu**, który ten próg produkuje, więc tabulatura jest powiązana z detekcją wysokości.

**P

6. Funkcja straty

Niech $\mathrm{valid}_t \in \{0, 1\}$ oznacza, że klatka t nie jest dopełnieniem krótkiego wycinka. Maski: $m_{\text{head}}(t, s) = \mathrm{valid}_t \cdot a_{\text{head}}$.

$$\mathcal{L}_{\text{on}} = \frac{\sum m_{\text{on}}\,\ell_{\alpha=4}(p_{\text{on}}, y_{\text{on}})}{\sum m_{\text{on}}}, \quad \mathcal{L}_{\text{fr}} = \frac{\sum m_{\text{fr}}\,\ell_{\alpha=1}(p_{\text{fr}}, y_{\text{fr}})}{\sum m_{\text{fr}}}, \quad \mathcal{L}_{\text{off}} = \frac{\sum m_{\text{off}}\,\ell_{\alpha=2}(p_{\text{off}}, y_{\text{off}})}{\sum m_{\text{off}}},$$

$$\mathcal{L}_{\text{vel}} = \frac{\sum \mu\,(\hat{v} - y_{\text{vel}})^2}{\max(1, \sum \mu)}, \quad \mu = y_{\text{vel_mask}} \cdot m_{\text{vel}}, \qquad \mathcal{L}_{\text{tab}} = \frac{1}{|\mathcal{V}|} \sum_{(t,i)\in\mathcal{V}} -\ln p_i(t)[c_i(t)],$$

gdzie ℓ_α to ważone BCE z M10, a $\mathcal{V}$ — zbiór par (klatka, struna) z znaną etykietą i $\text{valid}_t = 1$, $a_{\text{tab}} = 1$. Całość:

$$\boxed{\mathcal{L} = 1{,}0\,\mathcal{L}_{\text{on}} + 1{,}0\,\mathcal{L}_{\text{fr}} + 0{,}5\,\mathcal{L}_{\text{off}} + 0{,}5\,\mathcal{L}_{\text{vel}} + 0{,}5\,\mathcal{L}_{\text{tab}}}$$

(domyślnie bez straty ogniskowej: $\gamma = 0$, bez wygładzania etykiet.) Uwaga o głośności: **żadne źródło danych nie ma etykiet głośności**, więc $\sum \mu = 0$, $\mathcal{L}_{\text{vel}} = 0$ i głowica głośności pozostaje nieuczona (jej wyjście nie jest zwracane).

**P

7. Uczenie

- **Optymalizator:** AdamW, $\lambda = 10^{-4}$, przycinanie gradientu do normy 1, mieszana precyzja fp16 na GPU Tesla T4.
- **Paczka:** 16 losowych wycinków po 192 klatki (4,4 s). Źródło losowane z prawdopodobieństwami proporcjonalnymi do wag (klasyka 0,5; GuitarSet 0,3; środowiska 0,2), nagranie proporcjonalnie do długości.
- **Harmonogram:** rozgrzewka 1000 kroków, potem kosinus (M11). Sesja 1: $\eta_0 = 10^{-3} \to 0$, 36 epok × 1000 kroków. Sesja 3: restart $4 \cdot 10^{-4}$, dodane dane zestawu 2 i augmentacje (12 epok, zatrzymana przez wczesne zatrzymanie — błąd konfiguracji, jak odnotowano w dzienniku). Sesja 4: wznowienie od epoki 47, $3 \cdot 10^{-4} \to 0$ kosinusem od *czasu* (harmonogram kończy się dokładnie w terminie), 49 epok. Razem **97 epok**, 7,3 + 2,9 + 11 ≈ 21 h uczenia.
- **Wybór modelu:** epoka o najwyższym F1 nut na stałej próbie walidacyjnej 180 nagrań ze wszystkich źródeł: epoka 29 (89,42 %) w sesji 1, epoka **81 (90,27 %)** w sesji 4 — to jest model końcowy.
- **Augmentacje** (każda losowana niezależnie): zmiana poziomu ± 6 dB (dodanie $g/60$ do wartości znormalizowanych), szum gaussowski $\sigma = 0{,}02$, maskowanie pasm i chwil (SpecAugment), zmiana tempa $\times \exp(\mathcal{U}(-\ln 1{,}12, \ln 1{,}12))$ (losowa liczba z tego przedziału, każda równie prawdopodobna; zob. M0) z $p = 0{,}5$ przez rozciągnięcie wycinka i map, **rozstrojenie** (przesunięcie cech o ± 1 bin = ±33 ¢, etykiety bez zmian; $p = 0{,}3$), **niedokładne etykiety** (przesunięcie początków/końców o ±1 klatkę; $p = 0{,}3$), transpozycja o ±2 półtony ($p = 0{,}3$; tabulatura wtedy nieznana).

**P

8. Dekoder

Wejście: mapy $p_{\text{on}}, p_{\text{fr}}, p_{\text{off}}$ (prawdopodobieństwa), opcjonalnie $\hat{v}$ i tabulatura. Dla każdego półtonu s niezależnie, przeglądając klatki $t = 1..T$ (stan: początek `start`, ostatnia klatka `last`, licznik przerwy `gap`):

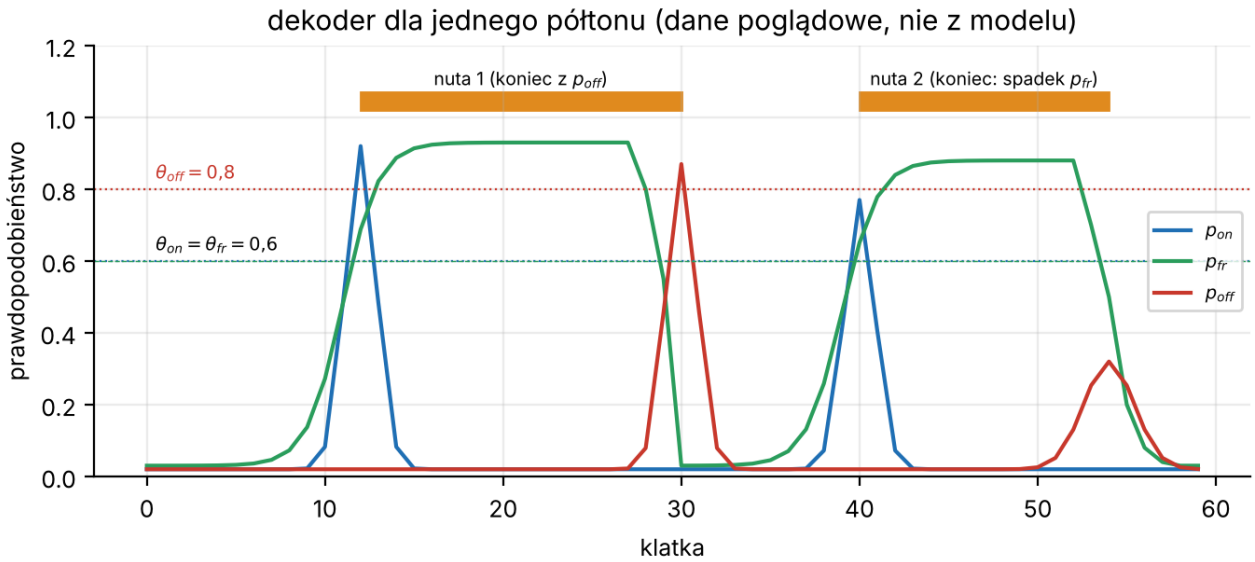
1. **Piki początków:** $\text{pk}_{\text{on}}(t, s) \Leftrightarrow$ lokalne maksimum $p_{\text{on}} \geq \theta_{\text{on}}$ (M12) **i** $p_{\text{fr}}(t, s) \geq \theta_{\text{on,fr}}$.

2. **Start nuty:** na piku początku rozpoczynamy nową nutę (jeśli trwała poprzednia, kończymy ją — to ponowne szarpnięcie). *Alternatywnie* (gdy nie ma piku) nuta startuje, jeśli $p_{\text{fr}} \geq \theta_{\text{start}}$ przez $n_{\min}$ kolejnych klatek.
3. **Trwanie:** dopóki $p_{\text{fr}}(t, s) \geq \theta_{\text{fr}}$, nuta trwa (`last = t`, `gap = 0`); w przeciwnym razie `gap +=` 1, a gdy `gap` $> g_{\max}$, nuta się kończy.
4. **Koniec z głowicy końca:** jeżeli t jest lokalnym maksimum $p_{\text{off}} \geq \theta_{\text{off}}$ i $t >$`start`, nuta kończy się w t.
5. **Wynik nuty:** wysokość $36 + s$; początek $t_0 = \max(0, \tau_{\text{start}})$; koniec $t_1 = \max\left(t_0 + d_{\min},\ \tau_{\text{last}} + \Delta/2\right)$; pewność = średnie p_{fr} w klatkach nuty; głośność $= \hat{v}(\text{start}, s)$; **struna**

$$i^* = \arg \max_{i:\, 0 \leq m - o_i < 20} \frac{1}{|\mathcal{N}|} \sum_{t \in \mathcal{N}} p_i(t) \big[\, 1 + (m - o_i) \,\big],$$

   czyli struna o największym średnim prawdopodobieństwie „gram właśnie ten próg” w klatkach nuty $\mathcal{N}$.
6. **Scalanie duplikatów:** dwie nuty tej samej wysokości o początkach bliższych niż θ_{merge} stają się jedną (dłuższą).

Nuty o mniej niż $n_{\min} = 2$ klatkach są odrzucane.



Rysunek 22: Działanie dekodera na jednym półtonie (kształty krzywych są poglądowe, nie pochodzą z modelu). Nuta 1 zaczyna się na piku p_{on} ($> 0{,}6$) i kończy na piku p_{off} ($> 0{,}8$). Nuta 2 ma za niski pik końca (0,3), więc kończy się dopiero, gdy p_{fr} spada poniżej 0,6 na dłużej niż $g_{\max} = 2$ klatki.

Przykład z liczbami. Dla E4 w klatkach 33–37 niech $p_{\text{on}} = (0{,}05;\ 0{,}30;\ 0{,}85;\ 0{,}40;\ 0{,}05)$ i $p_{\text{fr}} = (0{,}2;\ 0{,}7;\ 0{,}9;\ 0{,}9;\ 0{,}9)$. Klatka 35 spełnia $p_{\text{on}} = 0{,}85 \geq 0{,}6$, jest większa od obu sąsiadów (0,30 i 0,40), a $p_{\text{fr}} = 0{,}9 \geq 0{,}1$, więc to pik: początek nuty. Czas: $t_0 = \tau_{35} = 0{,}998$ s. Dalej $p_{\text{fr}} \geq 0{,}6$ trwa, więc nuta trwa. Gdy w klatce 53 $p_{\text{off}} = 0{,}9 \geq 0{,}8$ jest lokalnym maksimum, nuta kończy się: $t_1 = \tau_{53} + \Delta/2 = 1{,}4165 + 0{,}0116 = 1{,}428$ s. Mamy nutę (E4, 0,998 s – 1,428 s).

Przykład wyboru struny. Nuta MIDI 52 (E3) może być zagrana na strunie 6 (próg 12), 5 (próg 7) lub 4 (próg 2); struna 3 odpada ($52 - 55 < 0$). Jeśli średnie prawdopodobieństwa tych trzech

klas w klatkach nuty to $0{,}10;\ 0{,}20;\ 0{,}70$, dekoder wybiera strunę 4 (D) i próg 2.

Strojenie progów. Progi są hiperparametrami, nie parametrami uczonymi gradientem (dekoder nie jest różniczkowalny). Stroimy je na **walidacji** (222 nagrania: 12 GuitarSet, 150 klasyki, 60 środowisk) *przeszukiwaniem po współrzędnych*: po kolei dla każdego progu sprawdzamy siatkę wartości, zostawiamy najlepszą, powtarzamy dwa przebiegi. Cel:

$$J = 100 \cdot \frac{\sum_z w_z \, \overline{\tfrac{1}{2}\left(F_{\text{on}} + F_{\text{on+off}}\right)}_z}{\sum_z w_z}, \qquad w_{\text{GuitarSet}} = 0{,}4,\ w_{\text{klasyka}} = 0{,}4,\ w_{\text{środowiska}} = 0{,}2.$$

próg	domyślny	dostrojony
θ_{on} (próg początku)	0,5	**0,6**
$\theta_{\text{on,fr}}$ (wymagane p_{fr} przy początku)	0,3	**0,1**
θ_{fr} (trwanie)	0,5	**0,6**
θ_{off} (maksimum końca)	0,5	**0,8**
θ_{start} (nuta bez początku)	0,7	**1,01** (wyłączone)
n_{min} (min. długość, klatki)	2	2
g_{max} (przerwa, klatki)	1	**2**
θ_{merge} [s]	0,03	**0,08**
d_{min} [s]	0,06	0,06

Największy wkład dało wyłączenie nut bez wykrytego początku ($+0{,}75$ pkt celu); pozostałe zmiany — po 0,02–0,12 pkt, czyli na granicy szumu. Cel na walidacji wzrósł z 79,57 do 80,87.

**P

9. Protokół ewaluacji

Wszystkie miary (M13) są liczone przez ten sam moduł numpy, który przyjmuje listy nut, więc tak samo ocenia Gen 6 i wyniki innych generacji silnika. Implementacje porównano z `mir_eval` 0.8 z dokładnością 10^{-9} (testy automatyczne). Raportujemy:

- **F1 nut** (początek ±50 ms, wysokość ±50 ¢) — główna miara;
- **F1 nut z końcem** (dodatkowo koniec w tolerancji $\max(50\text{ ms}, 0{,}2\,\delta)$, gdzie δ to długość nuty wzorca w sekundach);
- **F1 klatkowe** (siatka 10 ms);
- mediana błędu początku, AP map, ECE, pokrycie czasu nuty, błąd oktawowy;
- tabulatura: F1 par (struna, próg), odsetek poprawnych strun, TDR (Wiggins i Kim 2019);
- statystyka **makro** (średnia po nagraniach) z 95 % przedziałem bootstrap po plikach oraz sparowany bootstrap przy porównaniach.

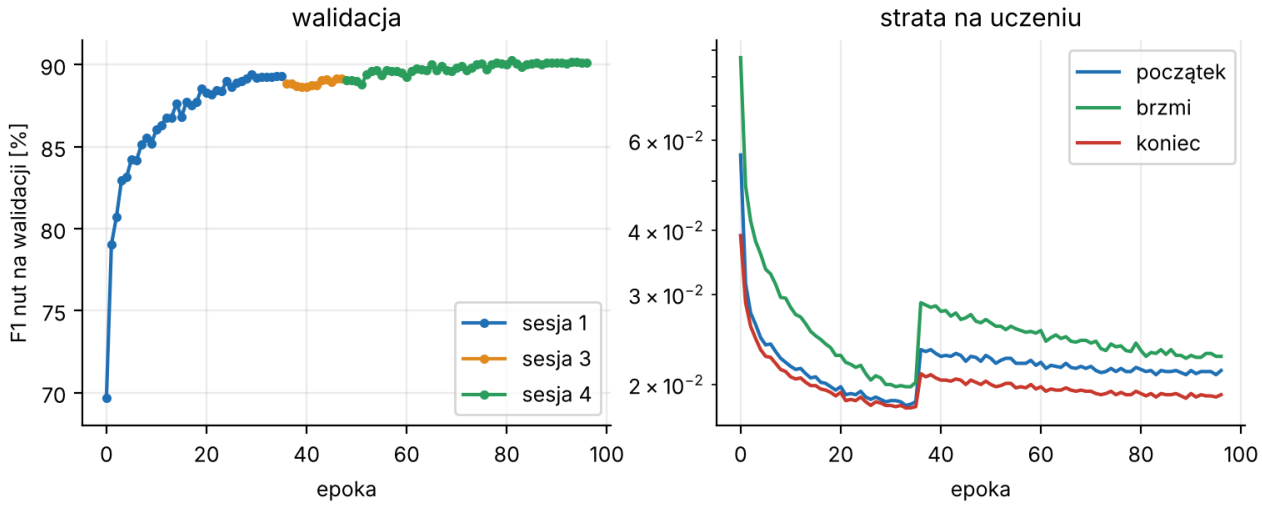
Zasada oddzielenia danych: progi dekodera stroiliśmy na walidacji; test (700 nagrań) nie brał udziału ani w uczeniu, ani w wyborze epoki, ani w strojeniu progów. Najlepszy próg początków obliczony *na teście* (0,82) jest podany tylko informacyjnie.

**P

10. Wyniki

10.1. Przebieg uczenia

F1 nut na walidacji (stała próba 180 nagrań): epoka 0: 69,7 %; epoka 5: 84,2 %; epoka 10: 86,1 %; epoka 20: 88,3 %; epoka 29: **89,42 %**; po restarcie i dodaniu danych 88,6–89,15 % (sesja 3, ostrożny spadek, bo dane są trudniejsze, a η wzrosło); sesja 4: 89,05 % (epoka 48) → 90,0–90,3 % w epokach 74–96, najlepsza **epoka 81: 90,27 %**. Strata tabulatury: 0,61 → 0,040 w sesji 1.



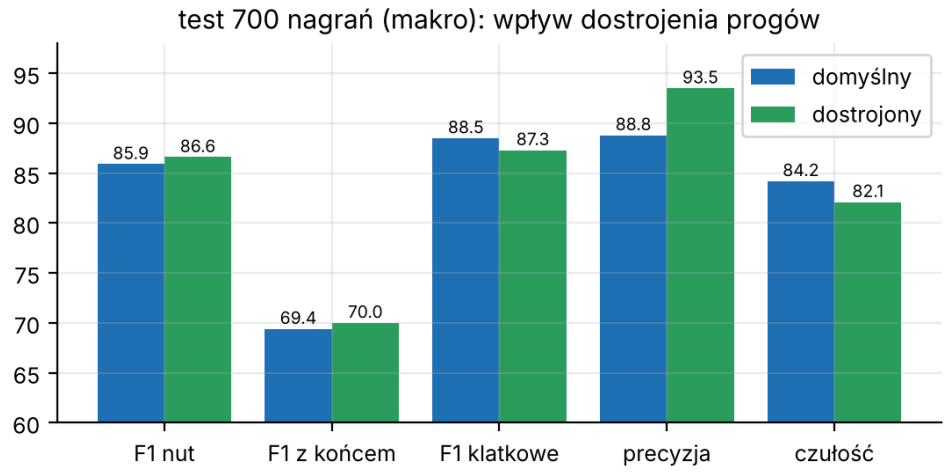
Rysunek 23: Lewo: F1 nut na walidacji w kolejnych epokach (trzy sesje). Widać wzrost z 69,7 % do ok. 90 %, spadek po restarcie w sesji 3 i dalszy wzrost w sesji 4. Prawo: strata na uczeniu (skala log). Dane: `docs/wyniki/epoki.csv`.

10.2. Test: model końcowy (epoka 81), 700 nagrań, makro

	domyślny dekoder	**dostrojony dekoder**	zmiana (sparowany bootstrap 95 %)
F1 nut	85,9 [85,0; 86,8]	**86,6**	+0,64 [0,46; 0,83]
F1 nut z końcem	69,4 [67,9; 71,0]	**70,0**	+0,59 [0,32; 0,85]
F1 klatkowe	88,5 [87,9; 89,1]	87,3	−1,29 [−1,50; −1,10]
precyzja / czułość nut	88,8 / 84,2	93,5 / 82,1	precyzja w górę, czułość w dół

Interpretacja: dostrojony dekoder jest ostrożniejszy — produkuje mniej fałszywych nut (precyzja +4,7 pkt), kosztem pomijania kilku prawdziwych (czułość −2,1 pkt). Dla listy nut (MIDI, tabulatura) to korzystny kompromis; miara klatkowa na tym traci, bo krótsze nuty pokrywają mniej czasu (pokrycie czasu nut 85,1 % → 81,1 %).

10.3. GuitarSet (prawdziwa gitara, gracze 03–05)



Rysunek 24: Wpływ dostrojenia progów dekodera na teście (makro, 700 nagrań): precyzja rośnie o 4,7 pkt, czułość spada o 2,1 pkt, F1 nut rośnie o 0,6 pkt, a F1 klatkowe spada o 1,3 pkt.

	Gen 5	Gen 6 (sesja 1)	Gen 6 końcowy	Gen 6 końcowy + dostrojony dekoder
F1 nut, razem	79,5	86,6	87,2	nie raportowano łącznie
akompaniament / melodie	75,6 / 83,4	84,4 / 88,8	85,1 / 89,2	**87,3 / 90,7**
nuty z końcem, razem	40,9	73,0	74,8	akompaniament 72,7, melodie 80,7

Tabulatura (końcowy model, domyślny dekoder, GuitarSet): akompaniament — F1 par struna-próg 82,2 %, poprawna struna 91,3 %, TDR 91,7 %; melodie — 75,1 %, 83,2 %, 84,6 %.

Liczby Gen 5 pochodzą z opisu projektu (ten sam podział GuitarSet 03–05). Dla dostrojonego dekodera dziennik podaje tylko podgrupy (akompaniament, melodie), stąd brak wartości łącznej.

10.4. Zróżnicowanie instrumentów (sesja 1, makro F1 nut)

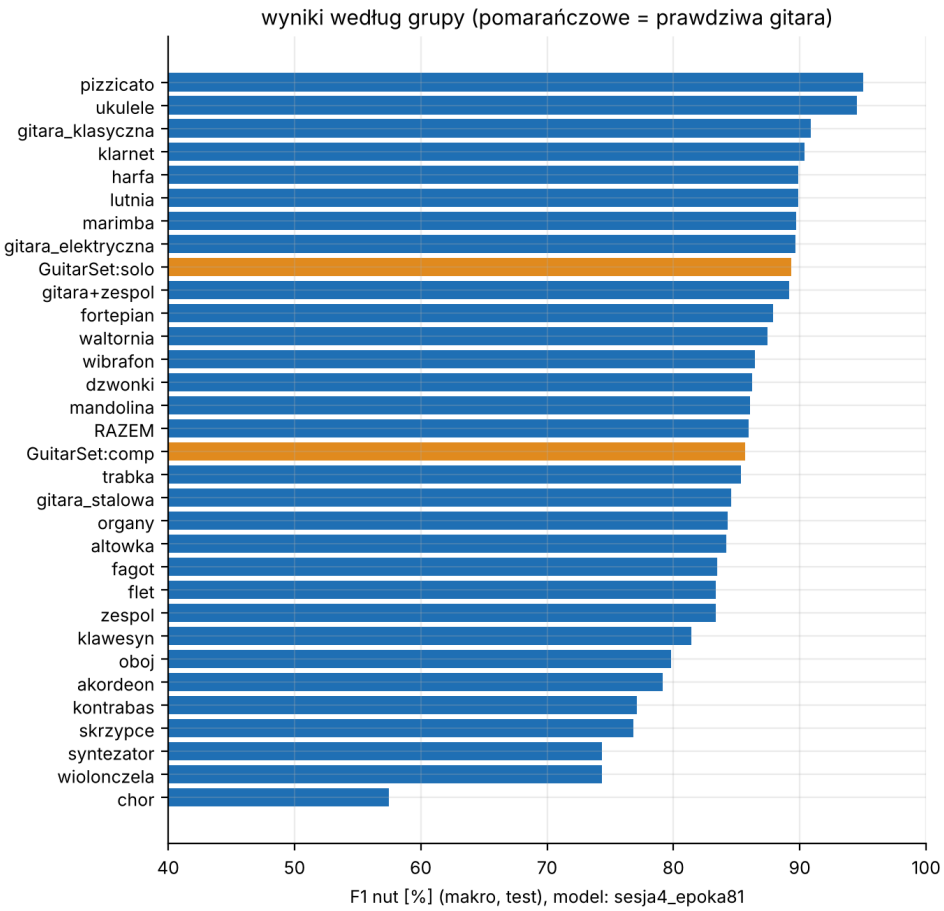
Najlepiej: ukulele 94,2 %, pizzicato 93,7 %, gitara elektryczna 90,0 %, klarnet 89,7 %, gitara klasyczna 89,9 %. Najsłabiej: chór 56,0 %, kontrabas 71,0 %, wiolonczela 71,3 %, syntezator 72,8 %, akordeon 74,6 %, skrzypce 75,4 %. Wzorzec jest zgodny z fizyką problemu: instrumenty o szarpniętym, wyraźnym początku (gitara, ukulele, pizzicato) są łatwe; instrumenty smyczkowe, wokalne i z wibrato mają rozmyty początek (tolerancja 50 ms jest trudna) i bogate alikwoty.

10.5. Szybkość

Nagranie 39 s, 4 rdzenie CPU: cechy 1,9 s + sieć 2,1 s ≈ 4,0 s, czyli **0,10 czasu nagrania**; 1 wątek: 0,30. Trzy minuty gry to ok. 19 s na 4 rdzeniach, 26–54 s na 1–2 rdzeniach.

11. Dyskusja i ograniczenia

1. **Dane syntetyczne.** Tylko GuitarSet (168 nagrań w uczeniu) jest prawdziwą gitarą; poprawa z kolejnych sesji (więcej danych syntetycznych) dała na GuitarSet najmniej (+0,4/+0,7 pkt). Dokładność na gitarze z innego mikrofonu, pokoju lub piezo jest niezmierzona; przewidziano dostrajanie na nagraniach piezo, brak jeszcze danych.
2. **Brak badań ablacyjnych.** Nie zmierzono osobno wpływu bloków harmonicznych, uwagi



Rysunek 25: F1 nut (makro) dla każdej grupy testowej, model końcowy (sesja 4, epoka 81). Pomarańczowe słupki to prawdziwa gitara (GuitarSet), niebieskie to nagrania syntetyczne. Dane: `docs/wyniki/test_grupy.csv`.

12. Podsumowanie

Gen 6 łączy cztery pomysły: (i) cechy HCQT z drobniejszym krokiem czasowym, (ii) *blok harmoniczny*, który widzi cały szereg alikwotów kandydata na wysokość, (iii) uwagę w czasie i między wysokościami, (iv) uczenie wielozadaniowe z maskami dostępności etykiet. Przy 249 tys. parametrów model osiąga makro-F1 nut 86,6 % na 700 niewidzianych nagraniach, a na GuitarSet 87,3 % (akompaniament) i 90,7 % (melodie), wobec 75,6 % i 83,4 % w poprzedniej generacji, przy czasie analizy 0,1 czasu nagrania na 4 rdzeniach. Główne otwarte problemy: błędy oktawowe, brak danych prawdziwej gitary, brak ablacji.

Najważniejsze myśli (podsumowanie dla uczącego się)

1. Dźwięk to ciąg próbek; wysokość mierzymy logarytmicznie (półtony, centy).
2. Każda nuta ma szereg alikwotów, który na osi logarytmicznej wygląda tak samo dla każdej wysokości.
3. HCQT = widmo w skali logarytmicznej z kanałem na każdy alikwot, w decybelach, znormalizowane do głośności.
4. Blok harmoniczny to filtr, który czyta siedem miejsc alikwotów naraz.
5. Uwaga w czasie mówi „co działo się wcześniej i później”, uwaga między wysokościami „co gra obok”.
6. Pięć głowic odpowiada na pięć pytań; tabulatura wybiera struny i progi softmaksem.
7. Strata to suma ważonych kar, z maskami dla brakujących etykiet.
8. AdamW z harmonogramem kosinusowym uczy wagi; progi dekodera strojono osobno na walidacji.
9. Jakość mierzymy F1 nut z tolerancją 50 ms i ćwierć tonu, z przedziałami bootstrap.
10. Model uzyskuje makro-F1 nut 86,6 % na teście; największe słabości to błędy oktawowe i niewielka ilość prawdziwej gitary w danych.

Literatura

1. Bittner, R. M., McFee, B., Salamon, J., Li, P., Bello, J. P. (2017). *Deep salience representations for F0 estimation in polyphonic music.* ISMIR.
2. Hawthorne, C., Elsen, E., Song, J., Roberts, A., Simon, I., Raffel, C., Engel, J., Oore, S., Eck, D. (2018). *Onsets and Frames: Dual-objective piano transcription.* ISMIR.
3. Xi, Q., Bittner, R. M., Pauwels, J., Ye, X., Bello, J. P. (2018). *GuitarSet: A dataset for guitar transcription.* ISMIR.
4. Wiggins, A., Kim, Y. (2019). *Guitar tablature estimation with a convolutional neural network.* ISMIR.
5. Vaswani, A. i in. (2017). *Attention is all you need.* NeurIPS.
6. Shaw, P., Uszkoreit, J., Vaswani, A. (2018). *Self-attention with relative position representations.* NAACL.
7. Ba, J. L., Kiros, J. R., Hinton, G. E. (2016). *Layer normalization.* arXiv:1607.06450.
8. Hendrycks, D., Gimpel, K. (2016). *Gaussian error linear units (GELUs).* arXiv:1606.08415.
9. Kingma, D. P., Ba, J. (2015). *Adam: A method for stochastic optimization.* ICLR.
10. Loshchilov, I., Hutter, F. (2019). *Decoupled weight decay regularization.* ICLR.

11. Loshchilov, I., Hutter, F. (2017). *SGDR: Stochastic gradient descent with warm restarts.* ICLR.
12. Yu, F., Koltun, V. (2016). *Multi-scale context aggregation by dilated convolutions.* ICLR.
13. Raffel, C., McFee, B., Humphrey, E. J., Salamon, J., Nieto, O., Liang, D., Ellis, D. P. W. (2014). *mir_eval: A transparent implementation of common MIR metrics.* ISMIR.
14. Poliner, G. E., Ellis, D. P. W. (2007). *A discriminative model for polyphonic piano transcription.* EURASIP J. Adv. Signal Process.
15. Hopcroft, J. E., Karp, R. M. (1973). *An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs.* SIAM J. Comput. 2(4).
16. Efron, B. (1979). *Bootstrap methods: Another look at the jackknife.* Ann. Statist. 7(1).
17. Guo, C., Pleiss, G., Sun, Y., Weinberger, K. Q. (2017). *On calibration of modern neural networks.* ICML.

Dodatek A. Skąd się bierze 249 219 parametrów

Konfiguracja końcowa: $H = 48$, $D = 64$, $n_h = 4$, $S = 53$, $W = 43$. Obliczone ręcznie wg wzoru z M6 i sprawdzone z liczbą podaną w dzienniku uczenia.

element	obliczenie	parametry
trzon: Conv 3×7, $9 \to 48$	$9 \cdot 48 \cdot 21 + 48$	9 120
blok harmoniczny (×3): LN(48)	$2 \cdot 48$	96
— Conv 1×1, $336 \to 48$	$336 \cdot 48 + 48$	16 176
— Depthwise 1×5	$48 \cdot 5 + 48$	288
— Conv 3×1, $48 \to 48$	$48 \cdot 48 \cdot 3 + 48$	6 960
— suma bloku × 3	$23\,520 \cdot 3$	70 560
zejście: Conv 1×3, $48 \to 64$	$48 \cdot 64 \cdot 3 + 64$	9 280
osadzenie wysokości	$53 \cdot 64$	3 392
warstwa kontekstu (×2): LN ×3	$3 \cdot 128$	384
— uwaga w czasie: W_{QKV}, W_O, premia	$12\,480 + 4\,160 + 4 \cdot 87$	16 988
— uwaga między wysokościami	$12\,480 + 4\,160 + 4 \cdot 105$	17 060
— FF: $64 \to 256 \to 64$	$16\,640 + 16\,448$	33 088
— suma warstwy × 2	$67\,520 \cdot 2$	135 040
LayerNorm końcowy	$2 \cdot 64$	128
głowica początku	$(64 \cdot 64 + 64) + (64 + 1)$	4 225
głowica „brzmi” (65 wejść)	$(65 \cdot 64 + 64) + 65$	4 289
głowica końca		4 225
głowica głośności		4 225
tabulatura: w_i (64→6), premie progów, MLP ciszy	$390 + 120 + 4\,225$	4 735
RAZEM		**249 219**

Dodatek B. Ściąga wzorów (jedną linijką)

co	wzór
nuta → częstotliwość	$f(m) = 440 \cdot 2^{(m-69)/12}$
odstęp w centach	$c = 1200 \log_2(f_2/f_1)$
alikwot w binach	$k_h = 36 \log_2 h$
rozdzielczość FFT	$\Delta f = f_s / N$
okno Hanna	$w[n] = \frac{1}{2}(1 - \cos(2\pi n/N))$
cecha HCQT	$\text{clip}\left((20 \log_{10} A - L_{\text{ref}})/60 + 1,\ 0,\ 1{,}3\right)$
sigmoid	$\sigma(z) = 1/(1 + e^{-z})$
softmax	$e^{z_i} / \sum_j e^{z_j}$
GELU	$z\Phi(z)$
LayerNorm	$\gamma(z - \mu)/\sqrt{\sigma^2 + \varepsilon} + \beta$
residual	$y = x + F(x)$
uwaga	$\text{softmax}\left(qk^\top/\sqrt{d_h} + b\right)v$
BCE ważona	$-[\alpha y \ln p + (1 - y) \ln(1 - p)]$
AdamW	$\theta \leftarrow \theta - \eta(\widehat{m}/(\sqrt{\widehat{v}} + \varepsilon) + \lambda\theta)$
kosinus	$\eta = \eta_0 \frac{1}{2}(1 + \cos \pi q)$
F1	$2TP/(2TP + FP + FN)$
tolerancja końca	$\max(50\,\text{ms},\ 0{,}2 \cdot \delta)$ (δ — długość nuty wzorca)
AP	$\sum_n (R_n - R_{n-1}) P_n$
ECE	$\sum_b \frac{\lvert\mathcal{B}_b\rvert}{N} \lvert\bar{y}_b - \bar{p}_b\rvert$
pole widzenia	$\text{halo} = 1 + \sum d_i \lfloor k/2 \rfloor + n_{\text{ctx}} \cdot 2W = 180$ klatek